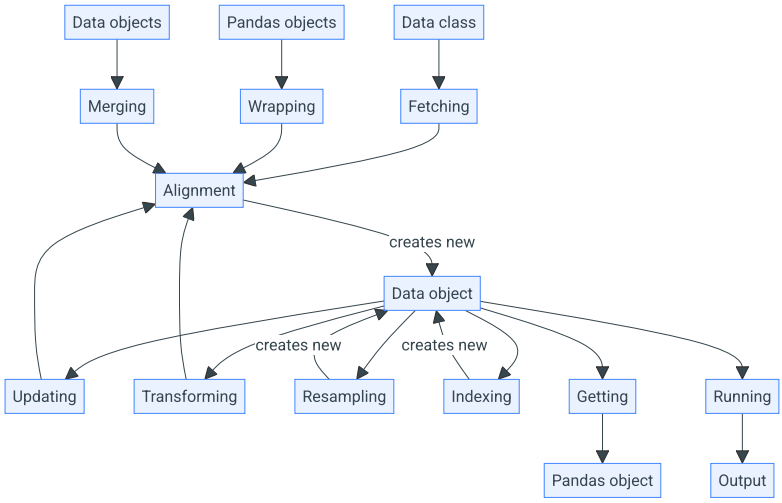
Documentation Data

Data

VectorBT PRO works on Pandas and NumPy arrays, but where those arrays are coming from? Getting the financial data manually is a challenging task, especially when an exchange can return only one bunch of data at a time such that iteration over time ranges, concatenation of results, and alignment of index and columns are effectively outsourced to the user. The task gets only trickier when multiple symbols are involved.

To simplify and automate data retrieval and management, vectorbt implements the **Data** class, which allows seamless handling of features (such as OHLC) and symbols (such as "BTC-USD"). It's a semi-abstract class, meaning you have to subclass it and define your own logic at various places to be able to use its rich functionality to the full extent. Gladly, there is a collection of custom data classes already implemented for us, but it's always good to know how to create such a data class on our own.

The steps discussed below can be visualized using the following graph:



(Reload the page if the diagram doesn't show up)

Fetching

Class **Data** implements an abstract class method **Data.fetch_symbol** for generating, loading, or fetching one symbol of data from any data source. It has to be overridden and implemented by the user, and return a single (Pandas or NumPy) array given some set of parameters, such as the starting date, the ending date, and the frequency.

Let's write a function that returns any symbol of data from Yahoo Finance using **yfinance**:

```
>>> from vectorbtpro import *

>>> def get_yf_symbol(symbol, period="max", start=None, end=None, **kwargs):
...     import yfinance as yf
...     if start is not None:
...         start = vbt.local_datetime(start)  1
...     if end is not None:
...         end = vbt.local_datetime(end)
...     return yf.Ticker(symbol).history(
...         period=period,
...         start=start,
...         end=end,
...         **kwargs
...     )

>>> get_yf_symbol("BTC-USD", start="2020-01-01", end="2020-01-05")
                                Open         High          Low        Close  \
Date
2019-12-31 00:00:00+00:00  7294.438965  7335.290039  7169.777832  7193.599121
2020-01-01 00:00:00+00:00  7194.892090  7254.330566  7174.944336  7200.174316
2020-01-02 00:00:00+00:00  7202.551270  7212.155273  6935.270020  6985.470215
2020-01-03 00:00:00+00:00  6984.428711  7413.715332  6914.996094  7344.884277
2020-01-04 00:00:00+00:00  7345.375488  7427.385742  7309.514160  7410.656738

                              Volume  Dividends  Stock Splits
Date
2019-12-31 00:00:00+00:00  21167946112        0.0           0.0
2020-01-01 00:00:00+00:00  18565664997        0.0           0.0
2020-01-02 00:00:00+00:00  20802083465        0.0           0.0
2020-01-03 00:00:00+00:00  28111481032        0.0           0.0
2020-01-04 00:00:00+00:00  18444271275        0.0           0.0
```

1. Convert to datetime using **to_datetime**

> **Info**
>
> Why the returned data starts from `2019-12-31` and not from `2020-01-01`? The provided start and end dates are defined in the local timezone and then converted into UTC. In the Europe/Berlin timezone, depending upon the time of the year, `2020-01-01` gets translated into `2019-12-31 22:00:00`, which is the date Yahoo Finance actually receives. To provide any date directly as a UTC date, append "UTC": `2020-01-01 UTC` or construct a proper **Timestamp** instance.

Managing data in a Pandas format is acceptable when we are dealing with one symbol, but what about multiple symbols? Remember how vectorbt wants us to provide each of the open price, high price, and other features as separate variables? Each of those variables must have symbols laid out as columns, which means that we would have to manually fetch all symbols and properly reorganize their data layout. Having some symbols with different index or columns would just add to our headache.

Luckily, there is a class method **Data.pull** that solves most of the issues related to iterating over, fetching, and merging symbols. It takes from one to multiple symbols, fetches each one with **Data.fetch_symbol**, puts it into a dictionary, and passes this dictionary to **Data.from_data** for post-processing and class instantiation.

Building upon our example, let's subclass **Data** and override the **Data.fetch_symbol** method to call our `get_yf_symbol` function:

```
>>> class YFData(vbt.Data):
...     @classmethod
...     def fetch_symbol(cls, symbol, **kwargs):
...         return get_yf_symbol(symbol, **kwargs)
```

> **Hint**
>
> You can replace `get_yf_symbol` with any other function that returns any array-like data!

That's it, `YFData` is now a full-blown data class capable of pulling data from Yahoo Finance and storing it:

```
>>> yf_data = YFData.pull(
...     ["BTC-USD", "ETH-USD"],
...     start="2020-01-01",
...     end="2020-01-05"
... )
```

Symbol 2/2

The pulled data is stored inside the **Data.data** dictionary with symbols being keys and values being Pandas objects returned by **Data.fetch_symbol**:

```
>>> yf_data.data["ETH-USD"]
                                Open        High         Low       Close  \
Date
2019-12-31 00:00:00+00:00  132.612274  133.732681  128.798157  129.610855
2020-01-01 00:00:00+00:00  129.630661  132.835358  129.198288  130.802002
2020-01-02 00:00:00+00:00  130.820038  130.820038  126.954910  127.410179
2020-01-03 00:00:00+00:00  127.411263  134.554016  126.490021  134.171707
2020-01-04 00:00:00+00:00  134.168518  136.052719  133.040558  135.069366

                              Volume  Dividends  Stock Splits
Date
2019-12-31 00:00:00+00:00   8936866397        0.0           0.0
2020-01-01 00:00:00+00:00   7935230330        0.0           0.0
2020-01-02 00:00:00+00:00   8032709256        0.0           0.0
2020-01-03 00:00:00+00:00  10476845358        0.0           0.0
2020-01-04 00:00:00+00:00   7430904515        0.0           0.0
```

Exception handling

If **Data.fetch_symbol** returned `None` or an empty Pandas object or NumPy array, the symbol will be skipped entirely. **Data.pull** will also catch any exception raised in **Data.fetch_symbol** and skip the symbol if the argument `skip_on_error` is True (it's False by default!), otherwise, it will abort the procedure.

Generally, it's the task of **Data.fetch_symbol** to handle issues. Whenever there is a lot of data points to fetch and the fetcher relies upon a loop to concatenate different data bunches together, the best approach is to show the user a warning whenever an exception is thrown and return the data fetched up to the most recent point in time, similarly to how this was implemented in **BinanceData** and **CCXTData**. In such a case, vectorbt will replace the missing data points with NaN or drop them altogether, and keep track of the last index. You can then wait until your connection is stable and re-fetch the missing data using **Data.update**.

Custom context

Along with the data, **Data.fetch_symbol** can also return a dictionary with custom keyword arguments acting as a context of the fetching operation. This context can later be accessed in the symbol dictionary **Data.returned_kwargs**. For instance, this context may include any information on why the fetching process failed, the length of the remaining data left to fetch, or which rows the fetched data represents when reading a local file (as implemented by **CSVData** for data updates).

Just for the sake of example, let's save the current timestamp:

```
>>> class YFData(vbt.Data):
...     @classmethod
...     def fetch_symbol(cls, symbol, **kwargs):
...         returned_kwargs = dict(timestamp=vbt.timestamp())
...         return get_yf_symbol(symbol, **kwargs), returned_kwargs
```

```
>>> yf_data = YFData.pull("BTC-USD", start="2020-01-01", end="2020-01-05")
>>> yf_data.returned_kwargs
symbol_dict({'BTC-USD': {'timestamp': Timestamp('2023-08-28 20:08:50.893763')}})
```

Info

symbol_dict is a regular dictionary where information is grouped by symbol.

Alignment

Like most classes that hold data, the class Data subclasses Analyzable, so we can perform Pandas indexing on the class instance itself to select rows and columns in all Pandas objects stored inside that instance. Doing a single Pandas indexing operation on multiple Pandas objects with different labels is impossible, so what happens if we fetched symbol data from different date ranges or with different columns? Whenever Data.pull passes the (unaligned) data dictionary to Data.from_data, it calls Data.align_data, which does the following:

1. Converts any array-like data into a Pandas object
2. Removes rows with duplicate indices apart from the latest one
3. Calls Data.prepare_tzaware_index to convert each object's index into a timezone-aware index using DataFrame.tz_localize and DataFrame.tz_convert
4. Calls Data.align_index to align the index labels of all objects based on some rule. By default, it builds the union of all indexes, sorts the resulting index, and sets the missing data points in any object to NaN.
5. Calls Data.align_columns to align the column labels of all objects based on some rule - a similar procedure to aligning indexes.
6. Having the same index and columns across all objects, it builds a wrapper
7. Finally, it passes all the prepared information to the class constructor for instantiation

Let's illustrate this workflow in practice:

```
>>> yf_data = YFData.pull(
...     ["BTC-USD", "ETH-USD"],
...     start=vbt.symbol_dict({  1
...         "BTC-USD": "2020-01-01",
...         "ETH-USD": "2020-01-03"
...     }),
...     end=vbt.symbol_dict({
...         "BTC-USD": "2020-01-03",
...         "ETH-USD": "2020-01-05"
...     })
... )
UserWarning: Symbols have mismatching index. Setting missing data points to NaN.
```

1 Use symbol_dict to specify any argument per symbol

Symbol 2/2

```
>>> yf_data.data["BTC-USD"]
                                Open         High          Low        Close  \
Date
2019-12-31 00:00:00+00:00  7294.438965  7335.290039  7169.777832  7193.599121
2020-01-01 00:00:00+00:00  7194.892090  7254.330566  7174.944336  7200.174316
2020-01-02 00:00:00+00:00  7202.551270  7212.155273  6935.270020  6985.470215
2020-01-03 00:00:00+00:00          NaN          NaN          NaN          NaN
2020-01-04 00:00:00+00:00          NaN          NaN          NaN          NaN

                                Volume  Dividends  Stock Splits
Date
2019-12-31 00:00:00+00:00  2.116795e+10        0.0           0.0
2020-01-01 00:00:00+00:00  1.856566e+10        0.0           0.0
2020-01-02 00:00:00+00:00  2.080208e+10        0.0           0.0
2020-01-03 00:00:00+00:00           NaN        NaN           NaN
2020-01-04 00:00:00+00:00           NaN        NaN           NaN

>>> yf_data.data["ETH-USD"]
                                Open        High         Low       Close  \
Date
2019-12-31 00:00:00+00:00         NaN         NaN         NaN         NaN
2020-01-01 00:00:00+00:00         NaN         NaN         NaN         NaN
2020-01-02 00:00:00+00:00  130.820038  130.820038  126.954910  127.410179
2020-01-03 00:00:00+00:00  127.411263  134.554016  126.490021  134.171707
2020-01-04 00:00:00+00:00  134.168518  136.052719  133.040558  135.069366

                                Volume  Dividends  Stock Splits
Date
2019-12-31 00:00:00+00:00           NaN        NaN           NaN
2020-01-01 00:00:00+00:00           NaN        NaN           NaN
2020-01-02 00:00:00+00:00  8.032709e+09        0.0           0.0
2020-01-03 00:00:00+00:00  1.047685e+10        0.0           0.0
2020-01-04 00:00:00+00:00  7.430905e+09        0.0           0.0
```

Notice how we ended up with the same index and columns across all Pandas objects. We can now use this data in any vectorbt function without fearing any indexing errors.

NaNs

If some rows are present in one symbol and are missing in another, vectorbt will raise a warning with the text "Symbols have mismatching index". By default, the missing rows will be replaced by NaN. To drop them or raise an error instead, use the `missing_index` argument:

```
>>> yf_data = YFData.pull(
...     ["BTC-USD", "ETH-USD"],
...     start=vbt.symbol_dict({  # (1)!
...         "BTC-USD": "2020-01-01",
...         "ETH-USD": "2020-01-03"
...     }),
...     end=vbt.symbol_dict({
...         "BTC-USD": "2020-01-03",
...         "ETH-USD": "2020-01-05"
...     }),
...     missing_index="drop"
... )
UserWarning: Symbols have mismatching index. Dropping missing data points.
```

Symbol 2/2

```
>>> yf_data.data["BTC-USD"]
                                Open         High         Low        Close  \
Date
2020-01-02 00:00:00+00:00  7202.55127  7212.155273  6935.27002  6985.470215

                              Volume  Dividends  Stock Splits
Date
2020-01-02 00:00:00+00:00  20802083465        0.0           0.0

>>> yf_data.data["ETH-USD"]
                                Open        High        Low       Close  \
Date
2020-01-02 00:00:00+00:00  130.820038  130.820038  126.95491  127.410179

                             Volume  Dividends  Stock Splits
Date
2020-01-02 00:00:00+00:00  8032709256        0.0           0.0
```

Updating

Updating is a regular fetching operation that can be used both to update the existing data points and to add new ones. It requires specifying the first timestamp or row index of the update, and assumes that the data points prior to this timestamp or row index remain unchanged.

Similarly to **Data.fetch_symbol**, updating must be manually implemented by overriding a method **Data.update_symbol**. In contrast to the fetcher, the updater is an **instance** method and can access the data fetched earlier. For instance, it can access the keyword arguments initially passed to the fetcher, accessible in the symbol dictionary **Data.fetch_kwargs**. Those arguments can be used as default arguments or be overridden by any argument passed directly to the updater. Every data instance has also a symbol dictionary **Data.last_index**, which holds the last fetched index per symbol. We can use this index as the starting point of the next update.

Let's build a new `YFData` class that can also perform updates to the stored data:

```
>>> class YFData(vbt.Data):
...     @classmethod
...     def fetch_symbol(cls, symbol, **kwargs):
...         return get_yf_symbol(symbol, **kwargs)
...
...     def update_symbol(self, symbol, **kwargs):
...         defaults = self.select_fetch_kwargs(symbol)  (1)
...         defaults["start"] = self.select_last_index(symbol)  (2)
...         kwargs = vbt.merge_dicts(defaults, kwargs)  (3)
...         return self.fetch_symbol(symbol, **kwargs)  (4)
```

1. Get keyword arguments initially passed to **Data.fetch_symbol** for this particular symbol
2. Override the default value for the starting date. Note that changing the keys won't affect **Data.fetch_kwargs**, but be careful with mutable values!
3. Override the default arguments with new arguments in `kwargs` using **merge_dicts**
4. Pass the final arguments to **Data.fetch_symbol**

Once the **Data.update_symbol** method is implemented, we can call the method **Data.update** to iterate over each symbol and update its data. Under the hood, this method also aligns the index and column labels of all the returned Pandas objects, appends the new data to the old data through concatenation along rows, and updates the last index of each symbol for the use in the next data update. Finally, it produces a new instance of **Data** by using **Configured.replace**.

> **Important**
>
> Updating data never overwrites the existing data instance but always returns a new instance. Remember that most classes in vectorbt are read-only to enable caching and avoid side effects.

First, we'll fetch the same data as previously:

```
>>> yf_data = YFData.pull(
...     ["BTC-USD", "ETH-USD"],
...     start=vbt.symbol_dict({
...         "BTC-USD": "2020-01-01",
...         "ETH-USD": "2020-01-03"
...     }),
...     end=vbt.symbol_dict({
```

```
...         "BTC-USD": "2020-01-03",
...         "ETH-USD": "2020-01-05"
...     })
... )
UserWarning: Symbols have mismatching index. Setting missing data points to NaN.
```

Symbol 2/2

Even though both DataFrames end with the same date, our `YFData` instance knows that the `BTC-USD` symbol is 2 rows behind the `ETH-USD` symbol:

```
>>> yf_data.last_index
symbol_dict({
    'BTC-USD': Timestamp('2020-01-02 00:00:00+0000', tz='UTC'),
    'ETH-USD': Timestamp('2020-01-04 00:00:00+0000', tz='UTC')
})
```

We can also access the keyword arguments passed to the initial fetching operation:

```
>>> yf_data.fetch_kwargs
symbol_dict({
    'BTC-USD': {'start': '2020-01-01', 'end': '2020-01-03'},
    'ETH-USD': {'start': '2020-01-03', 'end': '2020-01-05'}
})
```

The `start` argument of each symbol will be replaced by its respective entry in **Data.last_index**, while the `end` argument can be overridden by any date that we specify during the update.

Note

Without specifying the end date, vectorbt will update only the latest data point of each symbol.

Let's update both symbols up to the same date:

```
>>> yf_data_updated = yf_data.update(end="2020-01-06")  (1)

>>> yf_data_updated.data["BTC-USD"]
                                Open         High          Low        Close  \
Date
2019-12-31 00:00:00+00:00  7294.438965  7335.290039  7169.777832  7193.599121
2020-01-01 00:00:00+00:00  7194.892090  7254.330566  7174.944336  7200.174316
2020-01-02 00:00:00+00:00  7202.551270  7212.155273  6935.270020  6985.470215
2020-01-03 00:00:00+00:00  6984.428711  7413.715332  6914.996094  7344.884277
2020-01-04 00:00:00+00:00  7345.375488  7427.385742  7309.514160  7410.656738
2020-01-05 00:00:00+00:00  7410.451660  7544.497070  7400.535645  7411.317383

                                Volume  Dividends  Stock Splits
Date
2019-12-31 00:00:00+00:00  2.116795e+10        0.0           0.0
2020-01-01 00:00:00+00:00  1.856566e+10        0.0           0.0
2020-01-02 00:00:00+00:00  2.080208e+10        0.0           0.0
2020-01-03 00:00:00+00:00  2.811148e+10        0.0           0.0
2020-01-04 00:00:00+00:00  1.844427e+10        0.0           0.0
2020-01-05 00:00:00+00:00  1.972507e+10        0.0           0.0

>>> yf_data_updated.data["ETH-USD"]
                               Open        High         Low       Close  \
Date
2019-12-31 00:00:00+00:00         NaN         NaN         NaN         NaN
2020-01-01 00:00:00+00:00         NaN         NaN         NaN         NaN
2020-01-02 00:00:00+00:00  130.820038  130.820038  126.954910  127.410179
2020-01-03 00:00:00+00:00  127.411263  134.554016  126.490021  134.171707
2020-01-04 00:00:00+00:00  134.168518  136.052719  133.040558  135.069366
2020-01-05 00:00:00+00:00  135.072098  139.410202  135.045624  136.276779

                               Volume  Dividends  Stock Splits
Date
2019-12-31 00:00:00+00:00         NaN        NaN           NaN
2020-01-01 00:00:00+00:00         NaN        NaN           NaN
2020-01-02 00:00:00+00:00  8.032709e+09        0.0           0.0
2020-01-03 00:00:00+00:00  1.047685e+10        0.0           0.0
2020-01-04 00:00:00+00:00  7.430905e+09        0.0           0.0
2020-01-05 00:00:00+00:00  7.526675e+09        0.0           0.0
```

(1) Same date for both symbols

Each symbol has been updated separately based on their `last_index` value: the symbol `BTC-USD` has received new rows ranging from `2020-01-02` to `2020-01-05`, while the symbol `ETH-USD` has only received new rows between `2020-01-04` to `2020-01-05`. We can now see that both symbols have been successfully synced up to the same ending date:

```
>>> yf_data_updated.last_index
symbol_dict({
    'BTC-USD': Timestamp('2020-01-05 00:00:00+0000', tz='UTC'),
    'ETH-USD': Timestamp('2020-01-05 00:00:00+0000', tz='UTC')
})
```

If the last index of the data update lies before the current `last_index` (that is, we want to update any data in the middle), all the data after the new last index will be disregarded:

```
>>> yf_data_updated = yf_data_updated.update(start="2020-01-01", end="2020-01-02")

>>> yf_data_updated.data["BTC-USD"]
                               Open         High          Low        Close  \
Date
2019-12-31 00:00:00+00:00  7294.438965  7335.290039  7169.777832  7193.599121
2020-01-01 00:00:00+00:00  7194.892090  7254.330566  7174.944336  7200.174316

                               Volume  Dividends  Stock Splits
Date
2019-12-31 00:00:00+00:00  2.116795e+10        0.0           0.0
2020-01-01 00:00:00+00:00  1.856566e+10        0.0           0.0

>>> yf_data_updated.data["ETH-USD"]
                               Open        High         Low       Close  \
Date
2019-12-31 00:00:00+00:00  132.612274  133.732681  128.798157  129.610855
2020-01-01 00:00:00+00:00  129.630661  132.835358  129.198288  130.802002

                               Volume  Dividends  Stock Splits
Date
2019-12-31 00:00:00+00:00  8.936866e+09        0.0           0.0
2020-01-01 00:00:00+00:00  7.935230e+09        0.0           0.0
```

Note

The last data point of an update is considered to be the most up-to-date point, thus no data stored previously can come after it.

Concatenation

By default, the returned data instance contains the whole data - the old data with the new data concatenated together. To return only the updated data, disable `concat`:

```
>>> yf_data_new = yf_data.update(end="2020-01-06", concat=False)

>>> yf_data_new.data["BTC-USD"]
                               Open         High          Low        Close  \
Date
2020-01-02 00:00:00+00:00  7202.551270  7212.155273  6935.270020  6985.470215
2020-01-03 00:00:00+00:00  6984.428711  7413.715332  6914.996094  7344.884277
2020-01-04 00:00:00+00:00  7345.375488  7427.385742  7309.514160  7410.656738
2020-01-05 00:00:00+00:00  7410.451660  7544.497070  7400.535645  7411.317383

                               Volume  Dividends  Stock Splits
Date
2020-01-02 00:00:00+00:00  2.080208e+10        0.0           0.0
2020-01-03 00:00:00+00:00  2.811148e+10        0.0           0.0
2020-01-04 00:00:00+00:00  1.844427e+10        0.0           0.0
2020-01-05 00:00:00+00:00  1.972507e+10        0.0           0.0

>>> yf_data_new.data["ETH-USD"]
                               Open        High         Low       Close  \
Date
2020-01-02 00:00:00+00:00  130.820038  130.820038  126.954910  127.410179
2020-01-03 00:00:00+00:00  127.411263  134.554016  126.490021  134.171707
2020-01-04 00:00:00+00:00  134.168518  136.052719  133.040558  135.069366
2020-01-05 00:00:00+00:00  135.072098  139.410202  135.045624  136.276779

                               Volume  Dividends  Stock Splits
Date
2020-01-02 00:00:00+00:00  8.032709e+09        0.0           0.0
2020-01-03 00:00:00+00:00  1.047685e+10        0.0           0.0
2020-01-04 00:00:00+00:00  7.430905e+09        0.0           0.0
2020-01-05 00:00:00+00:00  7.526675e+09        0.0           0.0
```

The returned data instance skips two timestamps: `2019-12-31` and `2020-01-01`, which weren't changed during that update. But even though the symbol `ETH-USD` only received new rows between `2020-01-04` to `2020-01-05`, it contains the old data for `2020-01-02` and `2020-01-03` as well, why so? Those timestamps were updated in the `BTC-USD` dataset, and because the index across all symbols must be aligned, we need to include some old data to avoid setting NaNs.

Getting

After the data has been fetched and a new **Data** instance has been created, getting the data is straight-forward using the **Data.data** dictionary or the method **Data.get**.

```
>>> yf_data = YFData.pull(
...     ["BTC-USD", "ETH-USD"],
...     start="2020-01-01",
...     end="2020-01-05"
... )
```

Symbol 2/2

Get all features of one symbol of data:

```
>>> yf_data.get(symbols="BTC-USD")
                               Open         High          Low        Close  \
Date
```

```
2019-12-31 00:00:00+00:00  7294.438965  7335.290039  7169.777832  7193.599121
2020-01-01 00:00:00+00:00  7194.892090  7254.330566  7174.944336  7200.174316
2020-01-02 00:00:00+00:00  7202.551270  7212.155273  6935.270020  6985.470215
2020-01-03 00:00:00+00:00  6984.428711  7413.715332  6914.996094  7344.884277
2020-01-04 00:00:00+00:00  7345.375488  7427.385742  7309.514160  7410.656738

                               Volume  Dividends  Stock Splits
Date
2019-12-31 00:00:00+00:00  21167946112        0.0           0.0
2020-01-01 00:00:00+00:00  18565664997        0.0           0.0
2020-01-02 00:00:00+00:00  20802083465        0.0           0.0
2020-01-03 00:00:00+00:00  28111481032        0.0           0.0
2020-01-04 00:00:00+00:00  18444271275        0.0           0.0
```

Get specific features of one symbol of data:

```
>>> yf_data.get(features=["High", "Low"], symbols="BTC-USD")
                                High          Low
Date
2019-12-31 00:00:00+00:00  7335.290039  7169.777832
2020-01-01 00:00:00+00:00  7254.330566  7174.944336
2020-01-02 00:00:00+00:00  7212.155273  6935.270020
2020-01-03 00:00:00+00:00  7413.715332  6914.996094
2020-01-04 00:00:00+00:00  7427.385742  7309.514160
```

Get one feature of all symbols of data:

```
>>> yf_data.get(features="Close")
symbol                         BTC-USD     ETH-USD
Date
2019-12-31 00:00:00+00:00  7193.599121  129.610855
2020-01-01 00:00:00+00:00  7200.174316  130.802002
2020-01-02 00:00:00+00:00  6985.470215  127.410179
2020-01-03 00:00:00+00:00  7344.884277  134.171707
2020-01-04 00:00:00+00:00  7410.656738  135.069366
```

Notice how symbols have become columns in the returned DataFrame? This is the format so much loved by vectorbt.

Get multiple features of multiple symbols of data:

```
>>> open_price, close_price = yf_data.get(features=["Open", "Close"])  (1)
```

(1) Tuple with DataFrames, one per feature

Hint

As you might have noticed, vectorbt returns different formats depending upon when there is one or multiple features/symbols captured by the data instance. To produce a consisting format irrespective of the number of features/symbols, pass `features` / `symbols` as a list or any other collection.

For example, running `yf_data.get(features="Close")` when there is only one symbol will produce a Series instead of a DataFrame. To force vectorbt to always return a DataFrame, pass `features=["Close"]`.

Magnet features

Magnet features are features with case-insensitive names that the **Data** class knows how to detect and query. They include static features such as OHLCV, but also those that can be computed dynamically such as VWAP, HLC/3, OHLC/4, and returns. Each feature is also associated with an instance property that returns that feature for all symbols in a data instance. For example, to get the close price and returns:

```
>>> yf_data.close
symbol                         BTC-USD     ETH-USD
Date
2019-12-31 00:00:00+00:00  7193.599121  129.610855
2020-01-01 00:00:00+00:00  7200.174316  130.802002
2020-01-02 00:00:00+00:00  6985.470215  127.410179
2020-01-03 00:00:00+00:00  7344.884277  134.171707
2020-01-04 00:00:00+00:00  7410.656738  135.069366

>>> yf_data.returns
symbol                      BTC-USD   ETH-USD
Date
2019-12-31 00:00:00+00:00  0.000000  0.000000
2020-01-01 00:00:00+00:00  0.000914  0.009190
2020-01-02 00:00:00+00:00 -0.029819 -0.025931
2020-01-03 00:00:00+00:00  0.051452  0.053069
2020-01-04 00:00:00+00:00  0.008955  0.006690
```

Running

Thanks to the unambiguous nature of magnet features, we can use them in feeding many functions across vectorbt, and since most functions don't accept `data` directly but expect features such as `close` to be provided separately, there is an urgent need for a method that can recognize what a function wants and pass the data to it accordingly. Such a method is **Data.run**: it accepts a function, parses its arguments, and upon recognition of a magnet feature, simply forwards it. This is especially useful for quickly running indicators, which are recognized automatically by their names:

```
>>> yf_data.run("sma", 3)
```

```
<vectorbtpro.indicators.factory.talib.SMA at 0x7ff75218b5b0>
```

If there are multiple third-party libraries that have the same indicator name, it's advisable to also provide a prefix with the name of the library to avoid any confusion:

```
>>> yf_data.run("talib_sma", 3)
<vectorbtpro.indicators.factory.talib.SMA at 0x7ff752111070>
```

This method also accepts names of all the simulation methods available in **Portfolio**, such as **Portfolio.from_holding**:

```
>>> yf_data.run("from_holding")
<vectorbtpro.portfolio.base.Portfolio at 0x7ff767e18970>
```

Features and symbols

Class **Data** implements various dictionaries that hold data per symbol, but also methods that let us manipulate that data.

We can view the list of features and symbols using the **Data.features** and **Data.symbols** property respectively:

```
>>> yf_data.features
['Open', 'High', 'Low', 'Close', 'Volume', 'Dividends', 'Stock Splits']

>>> yf_data.symbols
['BTC-USD', 'ETH-USD']
```

Additionally, there is a flag **Data.single_key** that is True if this instance holds only one symbol of data (or feature in case the instance is feature-oriented!). This has implications on **Getting** as we discussed in the hints above.

```
>>> yf_data.single_key
False
```

Dicts

Each data instance holds at least 5 dictionaries:

1. **Data.data** with the Pandas objects
2. **Data.classes** with the classes
3. **Data.fetch_kwargs** with the keyword arguments passed to the fetcher
4. **Data.returned_kwargs** with the keyword arguments returned by the fetcher
5. **Data.last_index** with the last fetched index

Each dictionary is a regular dictionary of either the type **symbol_dict** (mostly when the instance is symbol-oriented) or **feature_dict** (mostly when the instance is feature-oriented).

```
>>> yf_data.last_index["BTC-USD"]
Timestamp('2020-01-04 00:00:00+0000', tz='UTC')
```

Important

Do not change the values of the above dictionaries in-place. Whenever working with keyword arguments, make sure to build a new dict after selecting a symbol: `dict(data.fetch_kwargs[symbol])` - this won't change the parent dict in case you want to modify the keyword arguments for some task.

Selecting

One or more symbols can be selected using **Data.select**:

```
>>> yf_data.select("BTC-USD")
<__main__.YFData at 0x7ff6a97f4b38>

>>> yf_data.select("BTC-USD").get()
                                Open         High          Low        Close  \
Date
2019-12-31 00:00:00+00:00  7294.438965  7335.290039  7169.777832  7193.599121
2020-01-01 00:00:00+00:00  7194.892090  7254.330566  7174.944336  7200.174316
2020-01-02 00:00:00+00:00  7202.551270  7212.155273  6935.270020  6985.470215
2020-01-03 00:00:00+00:00  6984.428711  7413.715332  6914.996094  7344.884277
2020-01-04 00:00:00+00:00  7345.375488  7427.385742  7309.514160  7410.656738

                              Volume  Dividends  Stock Splits
Date
2019-12-31 00:00:00+00:00  21167946112        0.0           0.0
2020-01-01 00:00:00+00:00  18565664997        0.0           0.0
2020-01-02 00:00:00+00:00  20802083465        0.0           0.0
2020-01-03 00:00:00+00:00  28111481032        0.0           0.0
2020-01-04 00:00:00+00:00  18444271275        0.0           0.0
```

The operation above produced a new `YFData` instance with only one symbol - `BTC-USD`.

Note

Updating the data in a child instance won't affect the parent instance we copied from because updating creates a new Pandas object. But changing the data in-place will also propagate the change to the parent instance. To make both instances fully independent, pass `copy_mode_="deep"` (see **Configured.replace**).

Info

If the instance is feature-oriented, this method will apply to features rather than symbols.

Renaming

Symbols can be renamed using **Data.rename**:

```
>>> yf_data.rename({
...     "BTC-USD": "BTC/USD",
...     "ETH-USD": "ETH/USD"
... }).get("Close")
symbol                      BTC/USD     ETH/USD
Date
2019-12-31 00:00:00+00:00  7193.599121  129.610855
2020-01-01 00:00:00+00:00  7200.174316  130.802002
2020-01-02 00:00:00+00:00  6985.470215  127.410179
2020-01-03 00:00:00+00:00  7344.884277  134.171707
2020-01-04 00:00:00+00:00  7410.656738  135.069366
```

Warning

Renaming symbols may (and mostly will) break their updating. Use this only for getting.

Info

If the instance is feature-oriented, this method will apply to features rather than symbols.

Classes

Classes come in handy when we want to introduce another level of abstraction over symbols, such as to further divide symbols into industries and sectors; this would allow us to analyze symbols within their classes, and entire classes themselves. Classes can be provided to the fetcher via the argument `classes`; they must be specified per symbol, unless there is only one class that should be applied to all symbols. In the end, they will be converted into a (multi-)index and stacked on top of symbol columns when getting the symbol wrapper using **Data.get_symbol_wrapper**. Each class can be either provided as a string (which will be stored under the class name `symbol_class`), or as a dictionary where keys are class names and values are class values:

```
>>> cls_yfdata = vbt.YFData.pull(
...     ["META", "GOOGL", "NFLX", "BAC", "WFC", "TLT", "SHV"],
...     classes=[
...         dict(class1="Equity", class2="Technology"),
...         dict(class1="Equity", class2="Technology"),
...         dict(class1="Equity", class2="Technology"),
...         dict(class1="Equity", class2="Financial"),
...         dict(class1="Equity", class2="Financial"),
...         dict(class1="Fixed Income", class2="Treasury"),
...         dict(class1="Fixed Income", class2="Treasury"),
...     ],
...     start="2010-01-01",
...     missing_columns="nan"
... )
>>> cls_yfdata.close
class1                        Equity                                \
class2                    Technology                      Financial
symbol                          META       GOOGL        NFLX        BAC
Date
2010-01-04 00:00:00-05:00        NaN   15.684434    7.640000  12.977036
2010-01-05 00:00:00-05:00        NaN   15.615365    7.358571  13.398854
2010-01-06 00:00:00-05:00        NaN   15.221722    7.617143  13.555999
...                              ...         ...         ...        ...
2023-08-24 00:00:00-04:00  286.750000  129.779999  406.929993  28.620001
2023-08-25 00:00:00-04:00  285.500000  129.880005  416.029999  28.500000
2023-08-28 00:00:00-04:00  290.260010  131.009995  418.059998  28.764999

class1                           Fixed Income
class2                               Treasury
symbol                         WFC         TLT         SHV
Date
2010-01-04 00:00:00-05:00  19.073046   62.717960   99.920975
2010-01-05 00:00:00-05:00  19.596645   63.123013   99.884712
2010-01-06 00:00:00-05:00  19.624567   62.278038   99.893806
...                              ...         ...         ...
2023-08-24 00:00:00-04:00  41.430000   94.910004  110.389999
2023-08-25 00:00:00-04:00  41.230000   95.220001  110.389999
2023-08-28 00:00:00-04:00  41.880001   95.320000  110.400002

[3436 rows x 7 columns]
```

Apart from feeding classes to the fetcher, we can also replace them in any existing data instance, which will return a new data instance:

```
>>> new_cls_yfdata = cls_yfdata.replace(
...     classes=vbt.symbol_dict({
...         "META": dict(class1="Equity", class2="Technology"),
...         "GOOGL": dict(class1="Equity", class2="Technology"),
...         "NFLX": dict(class1="Equity", class2="Technology"),
...         "BAC": dict(class1="Equity", class2="Financial"),
...         "WFC": dict(class1="Equity", class2="Financial"),
...         "TLT": dict(class1="Fixed Income", class2="Treasury"),
...         "SHV": dict(class1="Fixed Income", class2="Treasury"),
...     })
... )
```

Or by using **Data.update_classes**:

```
>>> new_cls_yfdata = cls_yfdata.update_classes(
...     class1=vbt.symbol_dict({
...         "META": "Equity",
...         "GOOGL": "Equity",
...         "NFLX": "Equity",
...         "BAC": "Equity",
...         "WFC": "Equity",
...         "TLT": "Fixed Income",
...         "SHV": "Fixed Income",
...     }),
...     class2=vbt.symbol_dict({
...         "META": "Technology",
...         "GOOGL": "Technology",
...         "NFLX": "Technology",
...         "BAC": "Financial",
...         "WFC": "Financial",
...         "TLT": "Treasury",
...         "SHV": "Treasury",
...     })
... )
```

Info

If the instance is feature-oriented and the dictionary with classes is of the type **feature_dict**, the classes will be applied to features rather than symbols.

Wrapping

We don't need data instances to work with vectorbt since the main objects of vectorbt's operation are Pandas and NumPy arrays, but sometimes it's much more convenient having all the data located under the same **Data** container because it can be managed (aligned, resampled, transformed, etc.) in a standardized way. To wrap any custom Pandas object with a **Data** class, we can use the class method **Data.from_data**, which can take either a single Pandas object (will be stored under the symbol `symbol`), a symbol dictionary consisting of multiple Pandas objects - one per symbol, or a feature dictionary consisting of multiple Pandas objects - one per feature.

The Series/DataFrame to be wrapped normally has columns associated with features such as OHLC as opposed to symbols such as `BTCUSDT`, for example:

```
>>> btc_df = pd.DataFrame({
...     "Open": [7194.89, 7202.55, 6984.42],
...     "High": [7254.33, 7212.15, 7413.71],
...     "Low": [7174.94, 6935.27, 6985.47],
...     "Close": [7200.17, 6985.47, 7344.88]
... }, index=vbt.date_range("2020-01-01", periods=3))
>>> btc_df
               Open     High      Low    Close
2020-01-01  7194.89  7254.33  7174.94  7200.17
2020-01-02  7202.55  7212.15  6935.27  6985.47
2020-01-03  6984.42  7413.71  6985.47  7344.88

>>> my_data = vbt.Data.from_data(btc_df)
>>> my_data.hlc3
2020-01-01 00:00:00+00:00    7209.813333
2020-01-02 00:00:00+00:00    7044.296667
2020-01-03 00:00:00+00:00    7248.020000
Freq: D, dtype: float64
```

We can also wrap multiple Pandas objects keyed by symbol:

```
>>> eth_df = pd.DataFrame({
...     "Open": [127.41, 134.16, 135.07],
...     "High": [134.55, 136.05, 139.41],
...     "Low": [126.49, 133.04, 135.04],
...     "Close": [134.17, 135.06, 136.27]
... }, index=vbt.date_range("2020-01-03", periods=3))  1
>>> eth_df
              Open    High     Low   Close
2020-01-03  127.41  134.55  126.49  134.17
2020-01-04  134.16  136.05  133.04  135.06
2020-01-05  135.07  139.41  135.04  136.27

>>> my_data = vbt.Data.from_data({"BTCUSDT": btc_df, "ETHUSDT": eth_df})
>>> my_data.hlc3
symbol                       BTCUSDT     ETHUSDT
2020-01-01 00:00:00+00:00  7209.813333         NaN
2020-01-02 00:00:00+00:00  7044.296667         NaN
2020-01-03 00:00:00+00:00  7248.020000  131.736667
2020-01-04 00:00:00+00:00          NaN  134.716667
```

```
2020-01-05 00:00:00+00:00          NaN  136.906667
```

1. Use different dates to demonstrate alignment

If our data happen to have symbols as columns, enable `columns_are_symbols`:

```
>>> hlc3_data = vbt.Data.from_data(my_data.hlc3, columns_are_symbols=True)
>>> hlc3_data.get()
symbol                     BTCUSDT     ETHUSDT
2020-01-01 00:00:00+00:00  7209.813333         NaN
2020-01-02 00:00:00+00:00  7044.296667         NaN
2020-01-03 00:00:00+00:00  7248.020000  131.736667
2020-01-04 00:00:00+00:00          NaN  134.716667
2020-01-05 00:00:00+00:00          NaN  136.906667
```

In this case, the instance will become feature-oriented, that is, the DataFrame above will be stored in a **feature_dict** and the behavior of symbols and features will be swapped across many methods. To make the instance symbol-oriented as in most of our examples, additionally pass `invert_data=True`.

Merging

As you might have already noticed, the process of aligning data is logically separated from the process of fetching data, enabling us to merge and align any data retrospectively.

Instead of storing and managing all symbols as a single monolithic entity, we can manage them separately and merge into one data instance whenever this is actually needed. Such an approach may be particularly useful when symbols are distributed over multiple data classes, such as a mixture of remote and local data sources. For this, we can use the class method **Data.merge**, which takes two or more data instances, merges their information, and forwards the merged information to **Data.from_data**:

```
>>> yf_data_btc = YFData.pull(
...     "BTC-USD",
...     start="2020-01-01",
...     end="2020-01-03"
... )
>>> yf_data_eth = YFData.pull(
...     "ETH-USD",
...     start="2020-01-03",
...     end="2020-01-05"
... )
>>> merged_yf_data = YFData.merge(yf_data_btc, yf_data_eth)
>>> merged_yf_data.close
symbol                     BTC-USD     ETH-USD
Date
2019-12-31 00:00:00+00:00  7193.599121         NaN
2020-01-01 00:00:00+00:00  7200.174316         NaN
2020-01-02 00:00:00+00:00  6985.470215  127.410179
2020-01-03 00:00:00+00:00          NaN  134.171707
2020-01-04 00:00:00+00:00          NaN  135.069366
UserWarning: Symbols have mismatching index. Setting missing data points to NaN.
```

The benefit of this method is that it not only merges different symbols across different data instances, but it can also merge Pandas objects corresponding to the same symbol:

```
>>> yf_data_btc1 = YFData.pull(
...     "BTC-USD",
...     start="2020-01-01",
...     end="2020-01-03"
... )
>>> yf_data_btc2 = YFData.pull(
...     "BTC-USD",
...     start="2020-01-05",
...     end="2020-01-07"
... )
>>> yf_data_eth = YFData.pull(
...     "ETH-USD",
...     start="2020-01-06",
...     end="2020-01-08"
... )
>>> merged_yf_data = YFData.merge(yf_data_btc1, yf_data_btc2, yf_data_eth)
>>> merged_yf_data.close
symbol                     BTC-USD     ETH-USD
Date
2019-12-31 00:00:00+00:00  7193.599121         NaN
2020-01-01 00:00:00+00:00  7200.174316         NaN
2020-01-02 00:00:00+00:00  6985.470215         NaN
2020-01-04 00:00:00+00:00  7410.656738         NaN
2020-01-05 00:00:00+00:00  7411.317383  136.276779
2020-01-06 00:00:00+00:00  7769.219238  144.304153
2020-01-07 00:00:00+00:00          NaN  143.543991
```

Subclassing

We called **Data** on the class `YFData`, which automatically creates an instance of that class. Having an instance of `YFData`, we can update the data the same way as we did before.But what if the data instances to be merged originate from different data classes? If we used `YFData` for merging **CCXTData** and **BinanceData** instances, we wouldn't be able to update the data objects anymore since the method `YFData.update_symbol` was implemented specifically for the symbols supported by Yahoo Finance.

In such case, either use **Data**, which will raise an error when attempting to update, or create a subclass of it to handle updates using different data providers (which is fairly easy if you know which symbol belongs to which data class - just call the respective `fetch_symbol` or `update_symbol` method):

```
>>> bn_data_btc = vbt.BinanceData.pull(
...     "BTCUSDT",
...     start="2020-01-01",
...     end="2020-01-04")
>>> bn_data_btc.close
Open time
2020-01-01 00:00:00+00:00    7200.85
2020-01-02 00:00:00+00:00    6965.71
2020-01-03 00:00:00+00:00    7344.96
Freq: D, Name: Close, dtype: float64

>>> ccxt_data_eth = vbt.CCXTData.pull(
...     "ETH/USDT",
...     start="2020-01-03",
...     end="2020-01-06")
>>> ccxt_data_eth.close
Open time
2020-01-03 00:00:00+00:00    134.35
2020-01-04 00:00:00+00:00    134.20
2020-01-05 00:00:00+00:00    135.37
Freq: D, Name: Close, dtype: float64

>>> class MergedData(vbt.Data):
...     @classmethod
...     def fetch_symbol(cls, symbol, **kwargs):
...         if symbol.startswith("BN_"):
...             return vbt.BinanceData.fetch_symbol(symbol[3:], **kwargs)
...         if symbol.startswith("CCXT_"):
...             return vbt.CCXTData.fetch_symbol(symbol[5:], **kwargs)
...         raise ValueError(f"Unknown symbol '{symbol}'")
...
...     def update_symbol(self, symbol, **kwargs):
...         fetch_kwargs = self.select_fetch_kwargs(symbol)
...         fetch_kwargs["start"] = self.select_last_index(symbol)
...         kwargs = vbt.merge_dicts(fetch_kwargs, kwargs)
...         return self.fetch_symbol(symbol, **kwargs)

>>> merged_data = MergedData.merge(
...     bn_data_btc,
...     ccxt_data_eth,
...     rename={
...         "BTCUSDT": "BN_BTCUSDT",
...         "ETH/USDT": "CCXT_ETH/USDT"
...     },
...     missing_columns="drop"
... )
UserWarning: Symbols have mismatching index. Setting missing data points to NaN.
UserWarning: Symbols have mismatching columns. Dropping missing data points.

>>> merged_data = merged_data.update(end="2020-01-07")
>>> merged_data.close
symbol                     BN_BTCUSDT  CCXT_ETH/USDT
Open time
2020-01-01 00:00:00+00:00     7200.85            NaN
2020-01-02 00:00:00+00:00     6965.71            NaN
2020-01-03 00:00:00+00:00     7344.96         134.35
2020-01-04 00:00:00+00:00     7354.11         134.20
2020-01-05 00:00:00+00:00     7358.75         135.37
2020-01-06 00:00:00+00:00     7758.00         144.15
```

We just created a flexible data class that can fetch, update, and manage symbols from multiple data providers. Great!

Resampling

As a subclass of **Wrapping**, each data instance stores the normalized metadata of all Pandas objects stored in that instance. This metadata can be used for resampling (i.e., changing the time frame) of all Pandas objects at once. Since many data classes, such as **CCXTData**, have a fixed feature layout, we can define the resampling function for each of their features in a special config called "feature config" (stored under **Data.feature_config**) and bind that config to the class itself for the use by all instances. Similar to field configs in **Records**, this config also can be attached to an entire data class or on any of its instances. Whenever a new instance is created, the config of the class is copied over such that rewriting it wouldn't affect the class config.

Here's, for example, how the feature config of **BinanceData** looks like:

```
>>> vbt.pprint(vbt.BinanceData.feature_config)
HybridConfig({
    'Quote volume': dict(
        resample_func=<function BinanceData.<lambda> at 0x7ff7648d7280>
    ),
    'Taker base volume': dict(
        resample_func=<function BinanceData.<lambda> at 0x7ff7648d7310>
    ),
    'Taker quote volume': dict(
        resample_func=<function BinanceData.<lambda> at 0x7ff7648d73a0>
    )
})
```

Wondering where are the resampling functions for all the OHLCV features? Those features are universal, and recognized and resampled automatically.

Let's resample the entire daily BTC/USD data from Yahoo Finance to the monthly frequency:

```
>>> full_yf_data = vbt.YFData.pull("BTC-USD")  (1)
>>> ms_yf_data = full_yf_data.resample("M")
>>> ms_yf_data.close
Date
2014-09-01 00:00:00+00:00      386.944000
2014-10-01 00:00:00+00:00      338.321014
2014-11-01 00:00:00+00:00      378.046997
                                     ...
2023-06-01 00:00:00+00:00    30477.251953
2023-07-01 00:00:00+00:00    29230.111328
2023-08-01 00:00:00+00:00    25995.177734
Freq: MS, Name: Close, Length: 108, dtype: float64
```

(1) Use the built-in Yahoo Finance class - it already knows how to resample features such as dividends

Since vectorbt works with custom target indexes just as well as with frequencies, we can provide a custom index to resample to:

```
>>> resampler = vbt.Resampler.from_date_range(
...     full_yf_data.wrapper.index,
...     start=full_yf_data.wrapper.index[0],
...     end=full_yf_data.wrapper.index[-1],
...     freq="Y",
...     silence_warnings=True
... )
>>> y_yf_data = full_yf_data.resample(resampler)
>>> y_yf_data.close
2014-12-31 00:00:00+00:00      426.619995
2015-12-31 00:00:00+00:00      961.237976
2016-12-31 00:00:00+00:00    12952.200195
2017-12-31 00:00:00+00:00     3865.952637
2018-12-31 00:00:00+00:00     7292.995117
2019-12-31 00:00:00+00:00    28840.953125
2020-12-31 00:00:00+00:00    47178.125000
2021-12-31 00:00:00+00:00    39194.972656
Freq: A-DEC, Name: Close, dtype: float64
```

Note

Whenever providing a custom index, vectorbt will aggregate all the values after each index entry. The last entry aggregates all the values up to infinity. See GenericAccessor.resample_to_index.

If a data class doesn't have a fixed feature layout, such as HDFData, we need to adapt the feature config to each **data instance** instead of setting it to the entire data class. For example, if we convert `bn_data_btc` to a generic Data instance:

```
>>> data_btc = vbt.Data.from_data(bn_data_btc.data, single_key=True)
>>> data_btc.resample("M")
ValueError: Cannot resample feature 'Quote volume'. Specify resample_func in feature_config.

>>> for k, v in vbt.BinanceData.feature_config.items():
...     data_btc.feature_config[k] = v
>>> data_btc.resample("M")
<vectorbtpro.data.base.Data at 0x7fc0dfce1630>
```

The same can be done with a single copy operation using Data.use_feature_config_of:

```
>>> data_btc = vbt.Data.from_data(bn_data_btc.data, single_key=True)
>>> data_btc.use_feature_config_of(vbt.BinanceData)
>>> data_btc.resample("M").close
Open time
2020-01-01 00:00:00+00:00    7344.96
Freq: MS, Name: Close, dtype: float64
```

Realignment

Similarly to resampling, realignment also changes the frequency of data, but in contrast to resampling, it doesn't aggregate the data but includes only the latest data point available at each step in the target index. It uses GenericAccessor.realign_opening for "open" and GenericAccessor.realign_closing for any other feature. This has two major use cases: aligning multiple symbols from different timezones to a single index, and upsampling data. Let's align symbols with different timings:

```
>>> data = vbt.YFData.pull(
...     ["BTC-USD", "AAPL"],
...     start="2020-01-01",
...     end="2020-01-04"
... )
>>> data.close
symbol                       BTC-USD       AAPL
Date
2020-01-01 00:00:00+00:00  7200.174316        NaN
2020-01-02 00:00:00+00:00  6985.470215        NaN
2020-01-02 05:00:00+00:00          NaN  73.249016
2020-01-03 00:00:00+00:00  7344.884277        NaN
2020-01-03 05:00:00+00:00          NaN  72.536888

>>> new_index = data.index.ceil("D").drop_duplicates()
>>> new_data = data.realign(new_index, ffill=False)
>>> new_data.close
```

```
symbol                        BTC-USD       AAPL
Date
2020-01-01 00:00:00+00:00  7200.174316        NaN
2020-01-02 00:00:00+00:00  6985.470215        NaN
2020-01-03 00:00:00+00:00  7344.884277  73.249016
2020-01-04 00:00:00+00:00          NaN  72.536888
```

Transforming

The main challenge in transforming any data is that each symbol must have the same index and columns because we need to concatenate them into one Pandas object later, thus any transformation operation must ensure that it's applied on each symbol in the same way. To enforce that, the method **Data.transform** concatenates the data across all symbols and features into one big DataFrame, and passes it to an UDF for transformation. Once transformed, the method splits the result back into multiple smaller Pandas objects - one per symbol, aligns them, creates a new data wrapper based on the aligned index and columns, and finally, initializes a new data instance.

Let's drop any row that contains at least one NaN:

```
>>> full_yf_data = YFData.pull(["BTC-USD", "ETH-USD"])
>>> full_yf_data.close.info()
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 3268 entries, 2014-09-17 00:00:00+00:00 to 2023-08-28 00:00:00+00:00
Freq: D
Data columns (total 2 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   BTC-USD  3268 non-null   float64
 1   ETH-USD  2119 non-null   float64
dtypes: float64(2)
memory usage: 76.6 KB

>>> new_full_yf_data = full_yf_data.transform(lambda df: df.dropna())
>>> new_full_yf_data.close.info()
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 2119 entries, 2017-11-09 00:00:00+00:00 to 2023-08-28 00:00:00+00:00
Freq: D
Data columns (total 2 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   BTC-USD  2119 non-null   float64
 1   ETH-USD  2119 non-null   float64
dtypes: float64(2)
memory usage: 49.7 KB
```

We can also decide to pass only one feature or symbol at a time by setting `per_feature=True` and `per_symbol=True` respectively. By enabling both arguments simultaneously, we can instruct vectorbt to pass only one feature and symbol combination as a Pandas Series at a time.

Analysis

Each data class subclasses **Analyzable**, which makes it analyzable and indexable.

Indexing

We can perform Pandas indexing on the data instance to select rows and columns in all fetched Pandas objects. Supported operations are `iloc`, `loc`, `xs`, and `[]`:

```
>>> sub_yf_data = yf_data.loc["2020-01-01":"2020-01-03"]  (1)
>>> sub_yf_data
<__main__.YFData at 0x7fa9a0012396>

>>> sub_yf_data = sub_yf_data[["Open", "High", "Low", "Close"]]  (2)
>>> sub_yf_data
<__main__.YFData at 0x7fa9a0032358>

>>> sub_yf_data.data["BTC-USD"]
                               Open         High          Low        Close
Date
2020-01-01 00:00:00+00:00  7194.892090  7254.330566  7174.944336  7200.174316
2020-01-02 00:00:00+00:00  7202.551270  7212.155273  6935.270020  6985.470215
2020-01-03 00:00:00+00:00  6984.428711  7413.715332  6914.996094  7344.884277

>>> sub_yf_data.data["ETH-USD"]
                               Open        High         Low       Close
Date
2020-01-01 00:00:00+00:00  129.630661  132.835358  129.198288  130.802002
2020-01-02 00:00:00+00:00  130.820038  130.820038  126.954910  127.410179
2020-01-03 00:00:00+00:00  127.411263  134.554016  126.490021  134.171707
```

(1) Select rows (in `loc` the second date is inclusive!). Returns a new data instance.

(2) Select columns. Returns a new data instance.

> **Note**
>
> Don't attempt to select symbols in this way - this notation is reserved for rows and columns only. Use **Data.select** instead.

> **Info**
>
> If the instance is feature-oriented, this method will apply to features rather than symbols.

Stats and plots

As with every **Analyzable** instance, we can compute and plot various properties of the data stored in the instance.

Very often, a simple call of **DataFrame.info** and **DataFrame.describe** on any of the stored Series or DataFrames is enough to print a concise summary:

```
>>> yf_data = YFData.pull(
...     ["BTC-USD", "ETH-USD"],
...     start=vbt.symbol_dict({
...         "BTC-USD": "2020-01-01",
...         "ETH-USD": "2020-01-03"
...     }),
...     end=vbt.symbol_dict({
...         "BTC-USD": "2020-01-03",
...         "ETH-USD": "2020-01-05"
...     })
... )
```

Symbol 2/2

```
>>> yf_data.data["BTC-USD"].info()
<class 'pandas.core.frame.DataFrame'>
DatetimeIndex: 5 entries, 2019-12-31 00:00:00+00:00 to 2020-01-04 00:00:00+00:00
Freq: D
Data columns (total 7 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   Open          3 non-null      float64
 1   High          3 non-null      float64
 2   Low           3 non-null      float64
 3   Close         3 non-null      float64
 4   Volume        3 non-null      float64
 5   Dividends     3 non-null      float64
 6   Stock Splits  3 non-null      float64
dtypes: float64(7)
memory usage: 320.0 bytes

>>> yf_data.data["BTC-USD"].describe()
              Open         High          Low        Close        Volume  \
count     3.000000     3.000000     3.000000     3.000000  3.000000e+00
mean   7230.627441  7267.258626  7093.330729  7126.414551  2.017856e+10
std      55.394933    62.577102   136.908963   122.105641  1.408740e+09
min    7194.892090  7212.155273  6935.270020  6985.470215  1.856566e+10
25%    7198.721680  7233.242920  7052.523926  7089.534668  1.968387e+10
50%    7202.551270  7254.330566  7169.777832  7193.599121  2.080208e+10
75%    7248.495117  7294.810303  7172.361084  7196.886719  2.098501e+10
max    7294.438965  7335.290039  7174.944336  7200.174316  2.116795e+10

       Dividends  Stock Splits
count        3.0           3.0
mean         0.0           0.0
std          0.0           0.0
min          0.0           0.0
25%          0.0           0.0
50%          0.0           0.0
75%          0.0           0.0
max          0.0           0.0
```

But since any data instance can capture multiple symbols, using **StatsBuilderMixin.stats** can provide us with information on symbols as well:

```
>>> yf_data.stats()
Start                   2019-12-31 00:00:00+00:00
End                     2020-01-04 00:00:00+00:00
Period                           5 days 00:00:00
Total Symbols                                  2
Null Counts: BTC-USD                          14
Null Counts: ETH-USD                          14
Name: agg_stats, dtype: object

>>> yf_data.stats(column="Volume")  (1)
Start                   2019-12-31 00:00:00+00:00
End                     2020-01-04 00:00:00+00:00
Period                           5 days 00:00:00
Total Symbols                                  2
Null Counts: BTC-USD                           2
Null Counts: ETH-USD                           2
Name: Volume, dtype: object
```

(1) Print the stats for the column `Volume` only

To plot the data, we can use the method **Data.plot**, which produces an OHLC(V) chart whenever the Pandas object is a DataFrame with regular price features, and a line chart otherwise. The former can plot only one symbol of data, while the latter can plot only one feature of data; both can be specified with the `symbol` and `feature` argument respectively.

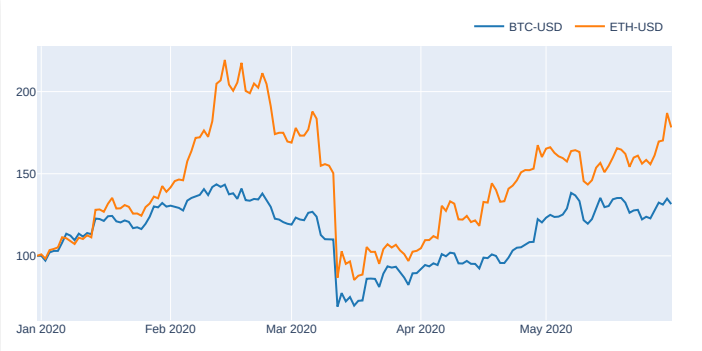
Since different symbols mostly have different starting values, we can provide an argument `base`, which will rebase the time series to start from the same point on chart:

```
>>> yf_data = YFData.pull(
...     ["BTC-USD", "ETH-USD"],
...     start="2020-01-01",
...     end="2020-06-01"
... )
```

Symbol 2/2

```
>>> yf_data.plot(column="Close", base=100).show()  (1)
```

(1) Since our data is symbol-oriented, `column` here is an alias for `feature`



Info

This only works for line traces since we cannot plot multiple OHLC(V) traces on the same chart.

Like most things, the same can be replicated using a chain of simple commands:

```
>>> yf_data["Close"].get().vbt.rebase(100).vbt.plot()  (1)
```

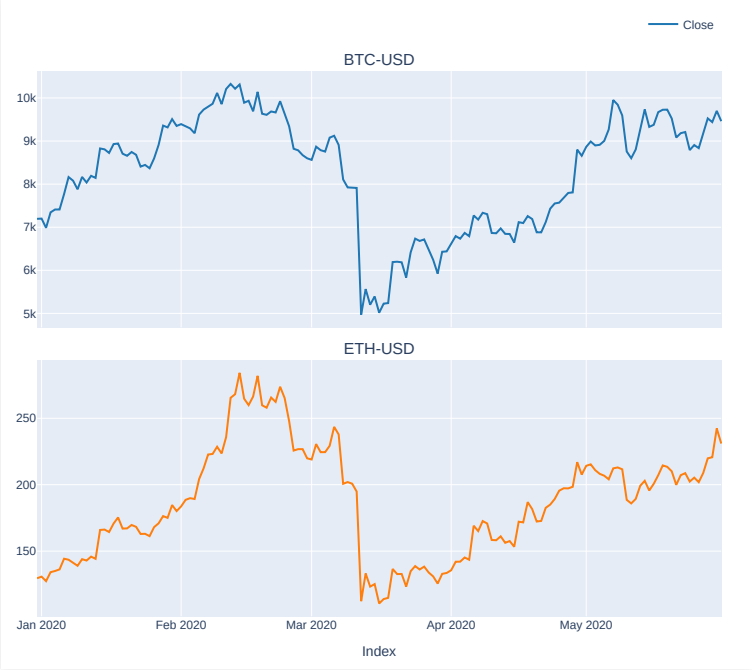
(1) Using **GenericAccessor.rebase** and **GenericAccessor.plot**

In addition, **Data** can display a subplot per symbol using **PlotsBuilderMixin.plots**, which utilizes **Data.plot** under the hood:

```
>>> yf_data.plots().show()
```

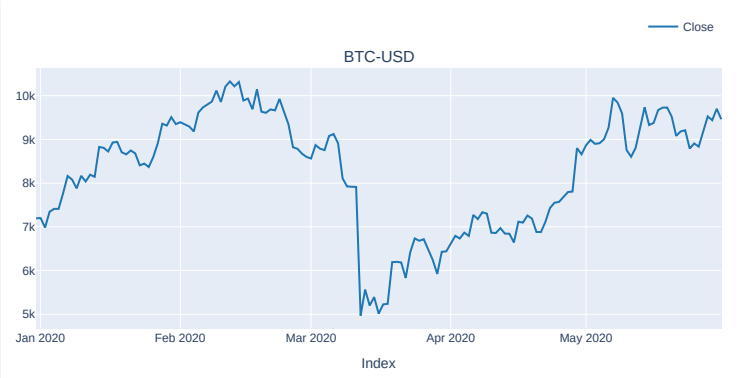
By also specifying a column, we can plot one feature per symbol of data:

```
>>> yf_data.plots(column="Close").show()
```



We can select one or more symbols by passing them via the `template_context` dictionary:

```
>>> yf_data.plots(
...     column="Close",
...     template_context=dict(symbols=["BTC-USD"])
... ).show()
```



If you look into the **Data.subplots** config, you'll notice only one subplot defined as a template. During the resolution phase, the template will be evaluated and the subplot will be expanded into multiple subplots - one per symbol - with the same name `plot` but prefixed with the index of that subplot in the expansion. For illustration, let's change the colors of both lines and plot their moving averages:

```
>>> from vectorbtpro.utils.colors import adjust_opacity

>>> fig = yf_data.plots(
...     column="Close",
...     subplot_settings=dict(
...         plot_0=dict(trace_kwargs=dict(line_color="mediumslateblue")),
...         plot_1=dict(trace_kwargs=dict(line_color="limegreen"))
...     )
... )
>>> sma = vbt.talib("SMA").run(yf_data.close, vbt.Default(10))  (1)
>>> sma["BTC-USD"].real.rename("SMA(10)").vbt.plot(
...     trace_kwargs=dict(line_color=adjust_opacity("mediumslateblue", 0.7)),
...     add_trace_kwargs=dict(row=1, col=1),  (2)
...     fig=fig
... )
>>> sma["ETH-USD"].real.rename("SMA(10)").vbt.plot(
...     trace_kwargs=dict(line_color=adjust_opacity("limegreen", 0.7)),
...     add_trace_kwargs=dict(row=2, col=1),
...     fig=fig
... )
>>> fig.show()
```

(1) Hide the `timeperiod` parameter from the column hierarchy by wrapping it with **Default**

(2) Specify the subplot to plot this SMA over

If you're hungry for a challenge, subclass the `YFData` class and override the **Data.plot** method such that it also runs and plots the SMA over the time series. This would make the plotting procedure ultra-flexible because now you can display the SMA for every feature and symbol without caring about the subplot's position and other things.

Python code

Documentation Data

Local

Repeatedly hitting remote API endpoints is costly, thus it's very important to cache data locally. Luckily, vectorbt implements a range of ways for managing local data.

Pickling

Like any other class subclassing **Pickleable**, we can save any **Data** instance to the disk using **Pickleable.save** and load it back using **Pickleable.load**. This will pickle the entire Python object including the stored Pandas objects, symbol dictionaries, and settings:

```
>>> from vectorbtpro import *

>>> yf_data = vbt.YFData.pull(
...     ["BTC-USD", "ETH-USD"],
...     start="2020-01-01",
...     end="2020-01-05"
... )
```

Symbol 2/2

```
>>> yf_data.save("yf_data")  (1)

>>> yf_data = vbt.YFData.load("yf_data")  (2)
>>> yf_data = yf_data.update(end="2020-01-06")
>>> yf_data.close
symbol                       BTC-USD     ETH-USD
Date
2020-01-01 00:00:00+00:00  7200.174316  130.802002
2020-01-02 00:00:00+00:00  6985.470215  127.410179
2020-01-03 00:00:00+00:00  7344.884277  134.171707
2020-01-04 00:00:00+00:00  7410.656738  135.069366
2020-01-05 00:00:00+00:00  7411.317383  136.276779
```

(1) Automatically adds the extension `.pickle` to the file name

(2) The object can be loaded back in a new runtime or even on another machine, just make sure to use a compatible vectorbt version

Important

The class definition won't be saved. If a new version of vectorbt introduces a breaking change to the **Data** constructor, the object may not load. In such a case, you can manually create a new instance:

```
>>> yf_data = vbt.YFData(**vbt.Configured.load("yf_data").config)
```

Saving

While pickling is a pretty fast and convenient solution to storing Python objects of any size, the pickled file is effectively a black box that requires a Python interpreter to be unboxed, which makes it unusable for many tasks since it cannot be imported by most other data-driven tools. To lift this limitation, the **Data** class allows us for saving exclusively the stored Pandas objects into one to multiple files of a tabular format.

CSV

The first supported file format is the **CSV** format, which is implemented by the instance method **Data.to_csv**. This method takes a path to the directory where the data should be stored (`path_or_buf`), and saves each symbol in a separate file using **DataFrame.to_csv**.

By default, it appends the extension `.csv` to each symbol, and saves the files into the current directory:

```
>>> yf_data.to_csv()
```

Info

Multiple symbols cannot be stored inside a single CSV file.

We can list all CSV files in the current directory using **list_files**:

```
>>> vbt.list_files("*.csv")
[PosixPath('ETH-USD.csv'), PosixPath('BTC-USD.csv')]
```

A cleaner approach is to save all the data in a separate directory:

```
>>> vbt.remove_file("BTC-USD.csv")  (1)
>>> vbt.remove_file("ETH-USD.csv")

>>> yf_data.to_csv("data", mkdir_kwargs=dict(mkdir=True))  (2)
```

1. Delete the CSV files created previously from the current directory
2. Save the files to the directory with the name `data`. If the directory doesn't exist, create a new one by passing keyword arguments `mkdir_kwargs` down to **check_mkdir**.

To save the data as tab-separated values (TSV):

```
>>> yf_data.to_csv("data", ext="tsv")

>>> vbt.list_files("data/*.tsv")
[PosixPath('data/BTC-USD.tsv'), PosixPath('data/ETH-USD.tsv')]
```

Hint

You don't have to pass `sep`: vectorbt will recognize the extension and pass the correct delimiter. But you can still override this argument if you need to split the data by a special character.

Similarly to **Data.pull**, we can provide any argument as a feature/symbol dictionary of type **key_dict** to define different rules for different symbols. Let's store the symbols from our example in separate directories:

```
>>> yf_data.to_csv(
...     vbt.key_dict({
...         "BTC-USD": "btc_data",
...         "ETH-USD": "eth_data"
...     }),
...     mkdir_kwargs=dict(mkdir=True)
... )
```

We can also specify the path to each file by using `path_or_buf` (first argument):

```
>>> yf_data.to_csv(
...     vbt.key_dict({
...         "BTC-USD": "data/btc_usd.csv",
...         "ETH-USD": "data/eth_usd.csv"
...     }),
...     mkdir_kwargs=dict(mkdir=True)
... )
```

To delete the entire directory (as part of a clean-up, for example):

```
>>> vbt.remove_dir("btc_data", with_contents=True)
>>> vbt.remove_dir("eth_data", with_contents=True)
>>> vbt.remove_dir("data", with_contents=True)
```

HDF

The second supported file format is the **HDF** format, which is implemented by the instance method **Data.to_hdf**. In contrast to **Data.to_csv**, this method can store multiple symbols inside a single file, where symbols are distributed as HDF keys.

By default, it creates a new file with the same name as the name of the data class and an extension `.h5`, and saves each symbol under a separate key in that file using **DataFrame.to_hdf**:

```
>>> yf_data.to_hdf()

>>> vbt.list_files("*.h5")
[PosixPath('YFData.h5')]
```

To see the list of all the groups and keys contained in an HDF file:

```
>>> with pd.HDFStore("YFData.h5") as store:
...     print(store.keys())
['/BTC-USD', '/ETH-USD']
```

Use the `key` argument to manually specify the key of a particular symbol:

```
>>> yf_data.to_hdf(
...     key=vbt.key_dict({
...         "BTC-USD": "btc_usd",
...         "ETH-USD": "eth_usd"
...     })
... )
```

Hint

If there is only one symbol, you don't need to use **key_dict**, just pass `key="btc_usd"`.

We can also specify the path to each file by using `path_or_buf` (first argument):

```
>>> yf_data.to_hdf(
...     vbt.key_dict({
...         "BTC-USD": "btc_usd.h5",
...         "ETH-USD": "eth_usd.h5"
...     })
```

```
... )
```

The arguments `path_or_buf` and `key` can be combined.

Any other argument behaves the same as for **Data.to_csv**.

Feather & Parquet

The third supported option is saving to a Feather/Parquet file, which is implemented by the instance method **Data.to_feather** and **Data.to_parquet** respectively. Feather is an unmodified raw columnar Arrow memory and is designed for short-term storage, while Parquet is often more expensive to write but features more layers of encoding and compression, making Parquet files often much smaller than Feather files. Another major difference between both is that we cannot partition data with Feather, neither we can natively store the index - it must be stored as a separate column, which is done automatically by vectorbt. We'll show to how to save to a Parquet memory.

By default, it appends the extension `.parquet` to each symbol, and saves the files into the current directory:

```
>>> yf_data.to_parquet()
```

Info

Multiple symbols cannot be stored inside a single Parquet file.

Other saving options are very similar to CSV saving options.

Apart from storing each DataFrame to a separate Parquet file, we can partition the DataFrame either by columns or rows. Partitioning by columns is controlled by the argument `partition_cols`, which takes a list of column names. Columns must be present in the data and are partitioned in the order they are given. Partitioning by rows is controlled by the argument `partition_by`, which takes a grouping instruction such as a frequency, an index (also multi-index), a Pandas grouper or resampler, or a vectorbt's own **Grouper** instance, and together with `groupby_kwargs` creates a new `Grouper` instance to be used in partitioning. Then, it attaches the groups as columns to each DataFrame, and provides the name of these columns as `partition_cols`. By default, if there's only one column, it will be named "group", and if there are more columns, they will be named after "group_{index}". To use own names, enable `keep_groupby_names`.

Info

When `partition_cols` or `partition_by` is provided, each symbol will be stored to a separate directory.

Let's partition our data by two days and save each DataFrame to its own Parquet directory:

```
>>> yf_data.to_parquet(partition_by="2 days")
```

Let's visualize the directory tree for the `BTC-USD` symbol:

```
>>> vbt.print_dir_tree("BTC-USD")
BTC-USD
├── group=2020-01-01%2000%3A00%3A00.000000000
│   └── 190335d948d04504b42a8d35ffb08f47-0.parquet
└── group=2020-01-03%2000%3A00%3A00.000000000
    └── 190335d948d04504b42a8d35ffb08f47-0.parquet

2 directories, 2 files
```

SQL

The fourth supported option is saving to a SQL database, which is implemented by the instance method **Data.to_sql**. It requires **SQLAlchemy** to be installed. This method takes the engine (object, name, or URL) and saves each symbol as a separate table to the database managed by the engine using `pd.to_sql`. The engine can be created either manually using SQLAlchemy's `create_engine` functon, or we can pass an URL and it will be created for us; in this case, it will be disposed at the end of the call.

Let's create a SQL database file in our working directory and store the data there:

```
>>> SQLITE_URL = "sqlite:///yf_data.db"

>>> yf_data.to_sql(SQLITE_URL)
```

If we want to continue working with the same engine and don't want to create another engine object, we can pass `return_engine=True` to return the engine object.

```
>>> engine = yf_data.to_sql(SQLITE_URL, if_exists="replace", return_engine=True)
```

We can also specify the schema by using the `schema` argument. Note, however, that some databases, such as SQLite, do not support the concept of a schema. If the schema doesn't exist, it will be created automatically.

```
>>> POSTGRESQL_URL = "postgresql://postgres:postgres@localhost:5432"

>>> yf_data.to_sql(POSTGRESQL_URL, schema="yf_data")
```

Use the `table` argument to manually specify the table name of a particular symbol:

```
>>> yf_data.to_sql(
...     POSTGRESQL_URL,
...     table=vbt.key_dict({
...         "BTC-USD": "BTC_USD",
...         "ETH-USD": "ETH_USD"
...     })
... )
```

Info

If the index is datetime-like and/or there are datetime-like columns, the method will localize/convert them to UTC first. To change this behavior, set the argument `to_utc` to False to deactivate, to "index" to apply it to the index only, and to "columns" to apply it to the columns only. In addition, the UTC timezone will be removed if `remove_utc_tz` is True (default) since some databases do not support timezone-aware timestamps; other timezones are not touched.

DuckDB

Hint

Previous method (using SQLAlchemy) can be used to write to a DuckDB database as well. For this, you have to install the **duckdb-engine** extension.

The fifth supported option is very similar to the previous one and allows to save data to a DuckDB database or CSV/Parquet/JSON file(s). It's implemented by the instance method **Data.to_duckdb**. It requires **DuckDB** to be installed. This method takes the connection (object or URL) and saves each symbol as a separate table to the database managed by the connection or to file(s) using a SQL query. The connection can be created either manually using DuckDB's `connect` funcion, or we can pass an URL and it will be created for us. If neither connection or connection-related keyword arguments are passed, the default in-memory connection is used.

Let's create a DuckDB database file in our working directory and store the data there:

```
>>> DUCKDB_URL = "database.duckdb"

>>> yf_data.to_duckdb(DUCKDB_URL)
```

We can also specify the catalog and schema by using the `catalog` and `schema` argument respectively. If the schema doesn't exist, it will be created automatically.

```
>>> yf_data.to_duckdb(DUCKDB_URL, schema="yf_data")
```

Use the `table` argument to manually specify the table name of a particular symbol:

```
>>> yf_data.to_duckdb(
...     DUCKDB_URL,
...     table=vbt.key_dict({
...         "BTC-USD": "BTC_USD",
...         "ETH-USD": "ETH_USD"
...     })
... )
```

There's also an option to save each DataFrame to a CSV, Parquet, or JSON file, rather than to the database itself. For this, we can use `write_format`, `write_path`, and `write_options`. The operation is performed using `COPY TO`. If `write_path` is a directory (which is the working directory by default), then each DataFrame will be saved to a file based on the specified format. Format is not needed if `write_path` points to a file with a recognizable extension. The options argument can be used to specify the options for writing; it can be either a string as in the DuckDB's documentation, such as `HEADER 1, DELIMITER ','`, or as a dictionary that will be translated into such a string by vectorbt, such as `dict(header=1, delimiter=',')`.

Let's save all data to Parquet files:

```
>>> yf_data.to_duckdb(
...     DUCKDB_URL,
...     write_path="data",
...     write_format="parquet",
...     mkdir_kwargs=dict(mkdir=True)
... )
```

Info

See the notes in the SQL section.

Loading

To import any previously stored data in a tabular format, we can either use Pandas or vectorbt's preset data classes specifically crafted for this job.

CSV

Each CSV dataset can be manually imported using **pandas.read_csv**:

```
>>> yf_data.to_csv()

>>> pd.read_csv("BTC-USD.csv", index_col=0, parse_dates=True)
                                Open         High          Low        Close  \
```

```
Date
2020-01-01 00:00:00+00:00  7194.892090  7254.330566  7174.944336  7200.174316
2020-01-02 00:00:00+00:00  7202.551270  7212.155273  6935.270020  6985.470215
2020-01-03 00:00:00+00:00  6984.428711  7413.715332  6914.996094  7344.884277
2020-01-04 00:00:00+00:00  7345.375488  7427.385742  7309.514160  7410.656738

                                Volume  Dividends  Stock Splits
Date
2020-01-01 00:00:00+00:00  18565664997        0.0           0.0
2020-01-02 00:00:00+00:00  20802083465        0.0           0.0
2020-01-03 00:00:00+00:00  28111481032        0.0           0.0
2020-01-04 00:00:00+00:00  18444271275        0.0           0.0
```

To join the imported datasets and wrap them with **Data**, we can use **Data.from_data**:

```
>>> btc_usd = pd.read_csv("BTC-USD.csv", index_col=0, parse_dates=True)
>>> eth_usd = pd.read_csv("ETH-USD.csv", index_col=0, parse_dates=True)

>>> data = vbt.Data.from_data({"BTC-USD": btc_usd, "ETH-USD": eth_usd})
>>> data.close
symbol                        BTC-USD     ETH-USD
Date
2020-01-01 00:00:00+00:00  7200.174316  130.802002
2020-01-02 00:00:00+00:00  6985.470215  127.410179
2020-01-03 00:00:00+00:00  7344.884277  134.171707
2020-01-04 00:00:00+00:00  7410.656738  135.069366
```

To relieve the user of the burden of manually searching, fetching, and merging CSV data, vectorbt implements the class **CSVData**, which can recursively explore directories for CSV files, resolve path expressions using **glob**, translate the found paths into symbols, and import and join tabular data - all automatically via a single command. It subclasses another class - **FileData**, which takes the whole credit for making all the above possible.

At the heart of the path matching functionality is the class method **FileData.pull**, which iterates over the specified paths, and for each one, finds the matching absolute paths using another class method **FileData.match_path**, and calls the abstract class method **FileData.fetch_key** to pull the data from the file located under that path.

Let's explore how **FileData.match_path** works by creating a directory with the name `data` and storing various empty files inside:

```
>>> vbt.make_dir("data", exist_ok=True)
>>> vbt.make_file("data/file1.csv")
>>> vbt.make_file("data/file2.tsv")
>>> vbt.make_file("data/file3")

>>> vbt.make_dir("data/sub-data", exist_ok=True)
>>> vbt.make_file("data/sub-data/file1.csv")
>>> vbt.make_file("data/sub-data/file2.tsv")
>>> vbt.make_file("data/sub-data/file3")
```

To get a better visual representation of the created contents:

```
>>> vbt.print_dir_tree("data")
data
├── file1.csv
├── file2.tsv
├── file3
└── sub-data
    ├── file1.csv
    ├── file2.tsv
    └── file3

1 directories, 6 files
```

Match all files in a directory:

```
>>> vbt.FileData.match_path("data")
[PosixPath("data/file1.csv"),
 PosixPath("data/file2.tsv"),
 PosixPath("data/file3")]
```

Match all CSV files in a directory:

```
>>> vbt.FileData.match_path("data/*.csv")
[PosixPath("data/file1.csv")]
```

Match all CSV files in a directory recursively:

```
>>> vbt.FileData.match_path("data/**/*.csv")
[PosixPath("data/file1.csv"), PosixPath("data/sub-data/file1.csv")]
```

For more details, see the documentation of **glob**.

Going back to **FileData.pull**: it can match one or multiple path expressions like above, provided either as `symbols` (if `paths` is None) or `paths`. Whenever we provide paths as symbols, the method calls **FileData.path_to_symbol** on each matched path to parse the name of the symbol (by default, it's the stem of the path):

```
>>> vbt.CSVData.pull("BTC-USD.csv").symbols
["BTC-USD"]

>>> vbt.CSVData.pull(["BTC-USD.csv", "ETH-USD.csv"]).symbols
```

```
["BTC-USD", "ETH-USD"]

>>> vbt.CSVData.pull("*.csv").symbols
["BTC-USD", "ETH-USD"]

>>> vbt.CSVData.pull(["BTC/USD", "ETH/USD"], paths="*.csv").symbols  (1)
["BTC/USD", "ETH/USD"]

>>> vbt.CSVData.pull(  (2)
...     ["BTC/USD", "ETH/USD"],
...     paths=["BTC-USD.csv", "ETH-USD.csv"]
... ).symbols
["BTC/USD", "ETH/USD"]
```

(1) Specify the symbols explicitly

(2) Specify the symbols and the paths explicitly

Note

Don't forget to filter by the `.csv`, `.tsv`, or any other extension in the expression.

Whenever we use a wildcard such as `*.csv`, vectorbt will sort the matched paths (per each path expression). To disable sorting, set `sort_paths` to False. We can also disable the path matching mechanism entirely by setting `match_paths` to False, which will forward all arguments directly to **CSVData.fetch_key**:

```
>>> vbt.CSVData.pull(
...     ["BTC/USD", "ETH/USD"],
...     paths=vbt.key_dict({
...         "BTC/USD": "BTC-USD.csv",
...         "ETH/USD": "ETH-USD.csv"
...     }),
...     match_paths=False
... ).symbols
["BTC/USD", "ETH/USD"]
```

Hint

Instead of paths, you can pass objects of any type supported by the `filepath_or_buffer` argument in **pandas.read_csv**.

To sum up the techniques discussed above, let's create an empty directory with the name `data` (again), write the `BTC-USD` symbol to a CSV file and the `ETH-USD` symbol to a TSV file, and load both datasets with a single `fetch` call:

```
>>> vbt.remove_dir("data", with_contents=True)

>>> yf_data.to_csv(
...     "data",
...     ext=vbt.key_dict({
...         "BTC-USD": "csv",
...         "ETH-USD": "tsv"
...     }),
...     mkdir_kwargs=dict(mkdir=True)
... )

>>> csv_data = vbt.CSVData.pull(["data/*.csv", "data/*.tsv"])  (1)
```

(1) The delimiter is recognized automatically based on the file's extension

Symbol 2/2

```
>>> csv_data.close
symbol                       BTC-USD     ETH-USD
Date
2020-01-01 00:00:00+00:00  7200.174316  130.802002
2020-01-02 00:00:00+00:00  6985.470215  127.410179
2020-01-03 00:00:00+00:00  7344.884277  134.171707
2020-01-04 00:00:00+00:00  7410.656738  135.069366
```

Note

Providing two paths with wildcards (`*`) doesn't mean we will get exactly two symbols: there may be more than one path matching each wildcard. You should imagine the two expressions above as being combined using the OR rule into a single expression `data/*.{csv,tsv}` (which isn't supported by **glob**, unfortunately).

The last but not the least is regex matching with `match_regex`, which instructs vectorbt to iterate over all matched paths and additionally validate them against a regular expression:

```
>>> vbt.CSVData.pull(
...     "data/**/*",  (1)
...     match_regex=r"^.*\.(csv|tsv)$"  (2)
... ).close
symbol                       BTC-USD     ETH-USD
Date
2020-01-01 00:00:00+00:00  7200.174316  130.802002
2020-01-02 00:00:00+00:00  6985.470215  127.410179
2020-01-03 00:00:00+00:00  7344.884277  134.171707
2020-01-04 00:00:00+00:00  7410.656738  135.069366
```

1. Recursively get the paths of all files in all subdirectories in `data`
2. Filter out any paths that do not end with `csv` or `tsv`

Any other argument is being passed directly to **CSVData.fetch_key** and then to **pandas.read_csv**.

Chunking

As an alternative to reading everything into memory, Pandas allows us to read data in chunks. In the case of CSV, we can load only a subset of lines into memory at any given time. Even though this is a very useful concept for processing big data, chunking doesn't provide many benefits when the only goal is to load the entire data into memory anyway.

Where chunking becomes really useful though is data filtering! The class **CSVData** as well as the function **pandas.read_csv** it's based on don't have arguments for skipping rows based on their content, only based on their row index. For example, to skip all the data that comes before `2020-01-03`, we would need to load the entire data into memory first. But once data becomes too large, we may run out of RAM. To account for this, we can split data into chunks and check the condition on each chunk at a time.

We have two options from here:

1. Use `chunksize` to split data into chunks of a fixed length
2. Use `iterator` to return an iterator that can be used to read chunks of a variable length

Both options make **pandas.read_csv** return an iterator of type `TextFileReader`. To make use of this iterator, **CSVData.fetch_key** accepts a user-defined function `chunk_func` that should 1) accept the iterator, 2) select, process, and concatenate chunks, and 3) return a Series or a DataFrame.

Let's fetch only those rows that have the date ending with an even day:

```
>>> csv_data = vbt.CSVData.pull(
...     ["data/*.csv", "data/*.tsv"],
...     chunksize=1,  (1)
...     chunk_func=lambda iterator: pd.concat([
...         df
...         for df in iterator
...         if (df.index.day % 2 == 0).all()
...     ], axis=0)
... )
```

1. Each chunk will be a DataFrame with one row

Symbol 2/2

```
>>> csv_data.close
symbol                       BTC-USD     ETH-USD
Date
2020-01-02 00:00:00+00:00  6985.470215  127.410179
2020-01-04 00:00:00+00:00  7410.656738  135.069366
```

> **Note**
>
> Chunking should mainly be used when memory considerations are more important than speed considerations.

HDF

Each HDF dataset can be manually imported using **pandas.read_hdf**:

```
>>> yf_data.to_hdf()

>>> pd.read_hdf("YFData.h5", key="BTC-USD")
                                 Open         High          Low        Close  \
Date
2020-01-01 00:00:00+00:00  7194.892090  7254.330566  7174.944336  7200.174316
2020-01-02 00:00:00+00:00  7202.551270  7212.155273  6935.270020  6985.470215
2020-01-03 00:00:00+00:00  6984.428711  7413.715332  6914.996094  7344.884277
2020-01-04 00:00:00+00:00  7345.375488  7427.385742  7309.514160  7410.656738

                               Volume  Dividends  Stock Splits
Date
2020-01-01 00:00:00+00:00  18565664997        0.0           0.0
2020-01-02 00:00:00+00:00  20802083465        0.0           0.0
2020-01-03 00:00:00+00:00  28111481032        0.0           0.0
2020-01-04 00:00:00+00:00  18444271275        0.0           0.0
```

Similarly to **CSVData** for CSV data, vectorbt implements a preset class **HDFData** tailored for reading HDF files. It shares the same parent class **FileData** and its fetcher **FileData.pull**. But in contrast to CSV datasets, which are stored one per file, HDF datasets are stored one per key in an HDF file. Since groups and keys follow the **POSIX**-style hierarchy with `/`-separators, we can query them in the same way as we query directories and files in a regular file system.

Let's illustrate this by using **HDFData.match_path**, which upgrades **FileData.match_path** with a proper discovery and handling of HDF groups and keys:

```
>>> vbt.HDFData.match_path("YFData.h5")
[PosixPath("YFData.h5/BTC-USD"), PosixPath("YFData.h5/ETH-USD")]
```

As we can see, the HDF file above is now being treated as a directory while groups and keys are being treated as subdirectories and files respectively. This makes importing HDF datasets as easy as CSV datasets:

```
>>> vbt.HDFData.pull("YFData.h5/BTC-USD").symbols  (1)
["BTC-USD"]

>>> vbt.HDFData.pull("YFData.h5").symbols  (2)
["BTC-USD", "ETH-USD"]

>>> vbt.HDFData.pull("YFData.h5/BTC*").symbols  (3)
["BTC-USD"]

>>> vbt.HDFData.pull("*.h5/BTC-*").symbols  (4)
["BTC-USD"]
```

(1) Matches the key `BTC-USD` in `YFData.h5`

(2) Matches all keys in `YFData.h5`

(3) Matches all keys starting with `BTC` in `YFData.h5`

(4) Matches all keys starting with `BTC` in all HDF files

Any other argument behaves the same as for **CSVData**, but now it's being passed directly to **HDFData.fetch_key** and then to **pandas.read_hdf**.

Chunking

Chunking for HDF files is identical to that for CSV files, but with two exceptions: the data must be saved as a **PyTables** Table structure by using `format="table"`, and the iterator is now of type `TableIterator` instead of `TextFileReader`.

```
>>> yf_data.to_hdf(format="table")

>>> hdf_data = vbt.HDFData.pull(
...     "YFData.h5",
...     chunksize=1,
...     chunk_func=lambda iterator: pd.concat([
...         df
...         for df in iterator
...         if (df.index.day % 2 == 0).all()
...     ], axis=0)
... )
```

Symbol 2/2

```
>>> hdf_data.close
symbol                       BTC-USD     ETH-USD
Date
2020-01-02 00:00:00+00:00  6985.470215  127.410179
2020-01-04 00:00:00+00:00  7410.656738  135.069366
```

Feather & Parquet

Each Parquet dataset can be manually imported using **pandas.read_parquet**:

```
>>> yf_data.to_parquet()

>>> pd.read_parquet("BTC-USD.parquet")
                                Open         High          Low        Close  \
Date
2020-01-01 00:00:00+00:00  7194.892090  7254.330566  7174.944336  7200.174316
2020-01-02 00:00:00+00:00  7202.551270  7212.155273  6935.270020  6985.470215
2020-01-03 00:00:00+00:00  6984.428711  7413.715332  6914.996094  7344.884277
2020-01-04 00:00:00+00:00  7345.375488  7427.385742  7309.514160  7410.656738

                               Volume  Dividends  Stock Splits
Date
2020-01-01 00:00:00+00:00  18565664997        0.0           0.0
2020-01-02 00:00:00+00:00  20802083465        0.0           0.0
2020-01-03 00:00:00+00:00  28111481032        0.0           0.0
2020-01-04 00:00:00+00:00  18444271275        0.0           0.0
```

Same goes for partitioned datasets:

```
>>> yf_data2 = vbt.YFData.pull(
...     ["BNB-USD", "XRP-USD"],
...     start="2020-01-01",
...     end="2020-01-05"
... )
>>> yf_data2.to_parquet(partition_by="2D")

>>> pd.read_parquet("BNB-USD")  (1)
                                Open       High        Low      Close  \
Date
2020-01-01 00:00:00+00:00  13.730962  13.873946  13.654942  13.689083
2020-01-02 00:00:00+00:00  13.698126  13.715548  12.989974  13.027011
2020-01-03 00:00:00+00:00  13.035329  13.763709  13.012638  13.660452
2020-01-04 00:00:00+00:00  13.667442  13.921914  13.560008  13.891512

                              Volume  Dividends  Stock Splits  \
Date
2020-01-01 00:00:00+00:00  172980718        0.0           0.0
2020-01-02 00:00:00+00:00  156376427        0.0           0.0
2020-01-03 00:00:00+00:00  173683857        0.0           0.0
```

```
2020-01-04 00:00:00+00:00  182230374        0.0           0.0

                                                    group
Date
2020-01-01 00:00:00+00:00  2020-01-01 00:00:00.000000000
2020-01-02 00:00:00+00:00  2020-01-01 00:00:00.000000000
2020-01-03 00:00:00+00:00  2020-01-03 00:00:00.000000000
2020-01-04 00:00:00+00:00  2020-01-03 00:00:00.000000000
```

(1) The path is pointing at a directory

Similarly to other classes for pulling data from local files, vectorbt implements preset classes **FeatherData** and **ParquetData** tailored for reading Feather and Parquet files respectively. They share the same parent class **FileData** and its fetcher **FileData.pull**.

But first, let's discover any Parquet files or directories stored in the current working directory using **ParquetData.list_paths**. What it does is searching for any files that have a ".parquet" extension as well as any directories with partitioned datasets that follow the **Hive partitioning scheme**.

```
>>> vbt.ParquetData.list_paths()
[PosixPath('BNB-USD'),
 PosixPath('BTC-USD.parquet'),
 PosixPath('ETH-USD.parquet'),
 PosixPath('XRP-USD')]

>>> vbt.ParquetData.is_parquet_file("BTC-USD.parquet")
True

>>> vbt.ParquetData.is_parquet_dir("BNB-USD")
True
```

This makes importing Parquet datasets as easy as any other file datasets:

```
>>> vbt.ParquetData.pull("BTC-USD.parquet").symbols  (1)
['BTC-USD']

>>> vbt.ParquetData.pull(["BTC-USD.parquet", "ETH-USD.parquet"]).symbols
['BTC-USD', 'ETH-USD']

>>> vbt.ParquetData.pull("*.parquet").symbols  (2)
['BTC-USD', 'ETH-USD']

>>> vbt.ParquetData.pull("BNB-USD").symbols  (3)
['BNB-USD']

>>> vbt.ParquetData.pull(["BNB-USD", "XRP-USD"]).symbols
['BNB-USD', 'XRP-USD']

>>> vbt.ParquetData.pull().symbols  (4)
['BNB-USD', 'BTC-USD', 'ETH-USD', 'XRP-USD']
```

(1) Pull a single-file dataset

(2) Pull all single-file datasets

(3) Pull a partitioned dataset

(4) Pull all single-file and partitioned datasets

But maybe you're wondering: what happens to that "group" column in partitioned datasets? Whenever a partitioned dataset is pulled and vectorbt sees one or more partition groups that are named "group" or "group_{index}", they are ignored automatically since they were most likely generated by using a user-defined row partitioning with `partition_by`. Such groups are also called default groups.

```
>>> vbt.ParquetData.list_partition_cols("BNB-USD")
['group']

>>> vbt.ParquetData.is_default_partition_col("group")
True

>>> vbt.ParquetData.pull("BNB-USD").features
['Open', 'High', 'Low', 'Close', 'Volume', 'Dividends', 'Stock Splits']
```

We can disable this behavior by setting `keep_partition_cols` to False:

```
>>> vbt.ParquetData.pull("BNB-USD", keep_partition_cols=True).features
['Open',
 'High',
 'Low',
 'Close',
 'Volume',
 'Dividends',
 'Stock Splits',
 'group']
```

SQL

Each SQL table can be manually imported using **pandas.read_sql_table**:

```
>>> pd.read_sql_table(
...     "BTC-USD",
...     POSTGRESQL_URL,
```

```
...     schema="yf_data",
...     index_col="Date",
...     parse_dates={"Date": dict(utc=True)}
... )
                                  Open         High          Low        Close  \
Date
2020-01-01 00:00:00+00:00  7194.892090  7254.330566  7174.944336  7200.174316
2020-01-02 00:00:00+00:00  7202.551270  7212.155273  6935.270020  6985.470215
2020-01-03 00:00:00+00:00  6984.428711  7413.715332  6914.996094  7344.884277
2020-01-04 00:00:00+00:00  7345.375488  7427.385742  7309.514160  7410.656738

                                Volume  Dividends  Stock Splits
Date
2020-01-01 00:00:00+00:00  18565664997        0.0           0.0
2020-01-02 00:00:00+00:00  20802083465        0.0           0.0
2020-01-03 00:00:00+00:00  28111481032        0.0           0.0
2020-01-04 00:00:00+00:00  18444271275        0.0           0.0
```

We can also execute any query using **pandas.read_sql_query**:

```
>>> pd.read_sql_query(
...     'SELECT * FROM yf_data."BTC-USD"',
...     POSTGRESQL_URL,
...     index_col="Date",
...     parse_dates={"Date": dict(utc=True)}
... )
                                  Open         High          Low        Close  \
Date
2020-01-01 00:00:00+00:00  7194.892090  7254.330566  7174.944336  7200.174316
2020-01-02 00:00:00+00:00  7202.551270  7212.155273  6935.270020  6985.470215
2020-01-03 00:00:00+00:00  6984.428711  7413.715332  6914.996094  7344.884277
2020-01-04 00:00:00+00:00  7345.375488  7427.385742  7309.514160  7410.656738

                                Volume  Dividends  Stock Splits
Date
2020-01-01 00:00:00+00:00  18565664997        0.0           0.0
2020-01-02 00:00:00+00:00  20802083465        0.0           0.0
2020-01-03 00:00:00+00:00  28111481032        0.0           0.0
2020-01-04 00:00:00+00:00  18444271275        0.0           0.0
```

But first, how do we know what kind of schemas and tables are stored in our database? We can call **SQLData.list_schemas** and **SQLData.list_tables** respectively. Most methods of **SQLData**, including these two, require the engine to be provided.

```
>>> vbt.SQLData.list_schemas(engine=POSTGRESQL_URL)
['information_schema', 'public', 'yf_data']

>>> vbt.SQLData.list_tables(engine=POSTGRESQL_URL)
['information_schema:_pg_foreign_data_wrappers',
 'information_schema:_pg_foreign_servers',
 ...
 'yf_data:BTC-USD',
 'yf_data:ETH-USD']

>>> vbt.SQLData.list_tables(engine=POSTGRESQL_URL, schema="yf_data")
['BTC-USD', 'ETH-USD']
```

To fetch the actual data, we have **SQLData.fetch_key** at our disposal, which calls `pd.read_sql_query` and does many pre-processings and post-processings under the hood. For instance, it can "inspect" the database and map columns indices to names, which aren't natively supported by the Pandas method. This is useful when specifying any information per column while relying solely on column positions, such as when providing `index_col` (which, by the way, defaults to 0 - the first column). It also does some heavy lifting on datetime index and columns, and can automatically retrieve all tables stored under a schema or even entire database.

Let's pull all the tables we stored previously, implicitly and explicitly:

```
>>> vbt.SQLData.pull(engine=SQLITE_URL).symbols  (1)
['BTC-USD', 'ETH-USD']

>>> vbt.SQLData.pull(["BTC-USD", "ETH-USD"], engine=SQLITE_URL).symbols  (2)
['BTC-USD', 'ETH-USD']

>>> vbt.SQLData.pull(engine=POSTGRESQL_URL, schema="yf_data").symbols  (3)
['BTC-USD', 'ETH-USD']

>>> vbt.SQLData.pull(["yf_data:BTC-USD", "yf_data:ETH-USD"], engine=POSTGRESQL_URL).symbols  (4)
['yf_data:BTC-USD', 'yf_data:ETH-USD']
```

1. Pull all tables from the database
2. Pull specific tables from the database
3. Pull all tables under a schema from the database
4. Pull tables from the database by also specifying the schema for each

We can also specialize any information per feature/symbol by providing it as an instance of **key_dict**:

```
>>> vbt.SQLData.pull(
...     ["BTCUSD", "ETHUSD"],
...     schema="yf_data",
...     table=vbt.key_dict({
...         "BTCUSD": "BTC-USD",
```

```
...         "ETHUSD": "ETH-USD",
...     }),
...     engine=POSTGRESQL_URL
... ).symbols
['BTCUSD', 'ETHUSD']
```

This is especially useful to execute custom queries:

```
>>> vbt.SQLData.pull(
...     ["BTC-USD", "ETH-USD"],
...     query=vbt.key_dict({
...         "BTC-USD": 'SELECT * FROM yf_data."BTC-USD"',
...         "ETH-USD": 'SELECT * FROM yf_data."ETH-USD"',
...     }),
...     index_col="Date",
...     engine=POSTGRESQL_URL
... ).symbols
['BTC-USD', 'ETH-USD']
```

Note

When executing a custom query, most of the preprocessings won't be available because the query isn't easily introspectable. For example, we have to provide a column name under `index_col` (or False to not use any column as an index).

Since different engines have different configurations and we don't want to repeatedly pass them during pulling, we can save the respective configuration to the global settings. For this, we need to create an engine name first and then save all the keyword arguments that we normally pass to **SQLData.fetch_key** under this engine name in `vbt.settings.data.engines`. This can be easily accomplished with **SQLData.set_engine_settings**, which takes an engine name and keyword arguments that should be saved. If the engine name is new, make sure to set `populate_` to True.

```
>>> vbt.SQLData.set_engine_settings(
...     engine_name="sqlite",
...     populate_=True,
...     engine=SQLITE_URL
... )

>>> vbt.SQLData.set_engine_settings(
...     engine_name="postgresql",
...     populate_=True,
...     engine=POSTGRESQL_URL,
...     schema="yf_data"
... )
```

If any argument during fetching is None, it will be looked under these settings first. All we have to do is to provide an engine name as `engine` or `engine_name`:

```
>>> vbt.SQLData.pull(engine_name="sqlite").symbols
['BTC-USD', 'ETH-USD']

>>> vbt.SQLData.pull(engine_name="postgresql").symbols
['BTC-USD', 'ETH-USD']
```

We can also save arguments that we want to use under each engine name. For example, let's define the default engine name:

```
>>> vbt.SQLData.set_custom_settings(engine_name="postgresql")

>>> vbt.SQLData.pull().symbols
['BTC-USD', 'ETH-USD']
```

To fetch some specific columns, we can use the `columns` argument:

```
>>> vbt.SQLData.pull(columns=["High", "Low"]).features
['High', 'Low']
```

Hint

If you want to pull just one column and keep it as a DataFrame, set `squeeze` to False.

In contrast to the Pandas method, we can also filter by any start and end condition. When `align_dates` is True (default) and `start` and/or `end` is provided, **SQLData.fetch_key** will fetch just one row of data first. It will then check whether the index of this row is datetime-like, and if so, will treat the provided `start` and/or `end` argument as a datetime-like object; this means converting it to a `datetime` object and localizing/converting it into the timezone of the index.

Note

If you used **Data.to_sql** to save the data, make sure to use the same `to_utc` option during pulling as during saving.

```
>>> vbt.SQLData.pull(start="2020-01-03").close
symbol                       BTC-USD     ETH-USD
Date
2020-01-03 00:00:00+00:00  7344.884277  134.171707
2020-01-04 00:00:00+00:00  7410.656738  135.069366
```

Most databases either don't support timezones, or data is stored in UTC, thus the default behavior is to localize any timezone-naive datetime to UTC; the user is then responsible to provide the correct timezone. If a timezone is provided via `tz` and the provided datetime has no timezone, it will be localized to `tz` and then

converted to the timezone of the index. If `to_utc` is True, it will be converted to UTC. If the index has no timezone, the provided datetime will be either converted to `tz` or UTC (if `to_utc` is True) first and then the timezone will be removed.

Let's demonstrate using a custom timezone by saving and then pulling the price of AAPL:

```
>>> aapl_data = vbt.YFData.fetch("AAPL", start="2022", end="2023")
>>> aapl_data.close
Date
2022-01-03 00:00:00-05:00    180.190979
2022-01-04 00:00:00-05:00    177.904068
2022-01-05 00:00:00-05:00    173.171844
                                ...
2022-12-28 00:00:00-05:00    125.504539
2022-12-29 00:00:00-05:00    129.059372
2022-12-30 00:00:00-05:00    129.378006
Name: Close, Length: 251, dtype: float64

>>> aapl_data.to_sql("sqlite")  (1)
>>> aapl_data.to_sql("postgresql")  (2)

>>> vbt.SQLData.pull(
...     "AAPL",
...     engine_name="sqlite",
...     start="2022-12-23",
...     end="2022-12-30",
...     tz="America/New_York"
... ).close
Date
2022-12-23 00:00:00-05:00    131.299820
2022-12-27 00:00:00-05:00    129.477585
2022-12-28 00:00:00-05:00    125.504539
2022-12-29 00:00:00-05:00    129.059372
Name: Close, dtype: float64

>>> vbt.SQLData.pull(
...     "AAPL",
...     engine_name="postgresql",
...     start="2022-12-23",
...     end="2022-12-30",
...     tz="America/New_York"
... ).close
Date
2022-12-23 00:00:00-05:00    131.299820
2022-12-27 00:00:00-05:00    129.477585
2022-12-28 00:00:00-05:00    125.504539
2022-12-29 00:00:00-05:00    129.059372
Name: Close, dtype: float64
```

(1) The first argument can also be an engine name from the global settings

(2) Schema defined in the global settings will be used by default

All datetime pre-processings can be disabled by turning off `align_dates`:

```
>>> vbt.SQLData.pull(
...     "AAPL",
...     start="2022-12-23",
...     end="2022-12-30",
...     tz="America/New_York",
...     align_dates=False
... ).close
Date
2022-12-23 00:00:00-05:00    131.299820
2022-12-27 00:00:00-05:00    129.477585
2022-12-28 00:00:00-05:00    125.504539
2022-12-29 00:00:00-05:00    129.059372
Name: Close, dtype: float64
```

When a custom query should be executed, the filtering must be done within the query:

```
>>> vbt.SQLData.pull(
...     "AAPL",
...     query="""
...         SELECT *
...         FROM yf_data."AAPL"
...         WHERE yf_data."AAPL"."Date" >= :start AND yf_data."AAPL"."Date" < :end
...     """,
...     tz="America/New_York",
...     index_col="Date",
...     params={
...         "start": vbt.datetime("2022-12-23", tz="America/New_York"),
...         "end": vbt.datetime("2022-12-30", tz="America/New_York")
...     }
... ).close
Date
2022-12-23 00:00:00-05:00    131.299820
2022-12-27 00:00:00-05:00    129.477585
2022-12-28 00:00:00-05:00    125.504539
2022-12-29 00:00:00-05:00    129.059372
Name: Close, dtype: float64
```

To filter by row number, we have to put a column with row numbers first. This can be done automatically by using **Data.to_sql** and setting `attach_row_number` to True.

```
>>> aapl_data.to_sql("sqlite", attach_row_number=True, if_exists="replace")
```

Then, when pulling, we can directly use `start_row` and `end_row`:

```
>>> vbt.SQLData.pull(
...     "AAPL",
...     start_row=5,
...     end_row=10,
...     tz="America/New_York"
... ).close
Date
2022-01-10 00:00:00-05:00    170.469116
2022-01-11 00:00:00-05:00    173.330231
2022-01-12 00:00:00-05:00    173.775711
2022-01-13 00:00:00-05:00    170.469116
2022-01-14 00:00:00-05:00    171.340347
Freq: D, Name: Close, dtype: float64
```

Chunking

Chunking for SQL databases is identical to that for CSV files:

```
>>> vbt.SQLData.pull(
...     "AAPL",
...     chunksize=1,
...     chunk_func=lambda iterator: pd.concat([
...         df
...         for df in iterator
...         if df.index.is_month_start.all()
...     ], axis=0),
...     tz="America/New_York"
... ).close
Date
2022-02-01 00:00:00-05:00    172.864914
2022-03-01 00:00:00-05:00    161.774826
2022-04-01 00:00:00-04:00    172.787796
2022-06-01 00:00:00-04:00    147.627930
2022-07-01 00:00:00-04:00    137.919113
2022-08-01 00:00:00-04:00    160.334793
2022-09-01 00:00:00-04:00    157.028458
2022-11-01 00:00:00-04:00    149.761551
2022-12-01 00:00:00-05:00    147.679932
Name: Close, dtype: float64
```

DuckDB

Hint

Previous method (using SQLAlchemy) can be used to read from a DuckDB database as well. For this, you have to install the **duckdb-engine** extension.

To fetch data using DuckDB, we can use the convenient class **DuckDBData**. It contains methods for discovering catalogs, schemas, and tables, as well as methods for fetching the actual data, such as **DuckDBData.fetch_key**. Let's delete the previous DuckDB database, create a new database by setting its URL globally, put some data inside, and take a look at the stored objects:

```
>>> vbt.remove_file(DUCKDB_URL)

>>> vbt.DuckDBData.set_custom_settings(
...     connection=DUCKDB_URL
... )

>>> yf_data.to_duckdb(schema="yf_data")

>>> vbt.DuckDBData.list_catalogs()
['database']

>>> vbt.DuckDBData.list_schemas()
['main', 'yf_data']

>>> vbt.DuckDBData.list_tables()
['yf_data:BTC-USD', 'yf_data:ETH-USD']
```

To fetch the data, we can use **DuckDBData.pull**. Whenever we provide one or more keys, they are automatically used as table names. Without any arguments, the method first determines the tables stored in the database and then pulls them.

```
>>> vbt.DuckDBData.pull("yf_data:BTC-USD").symbols
['yf_data:BTC-USD']

>>> vbt.DuckDBData.pull("BTC-USD", schema="yf_data").symbols
['BTC-USD']

>>> vbt.DuckDBData.pull(["BTC-USD", "ETH-USD"], schema="yf_data").symbols
['BTC-USD', 'ETH-USD']

>>> vbt.DuckDBData.pull().symbols
['yf_data:BTC-USD', 'yf_data:ETH-USD']
```

For cases where symbol names and table names should be kept separate, we can provide each table explicitly using a dictionary of type **key_dict**:

```
>>> vbt.DuckDBData.pull(
...     ["BTC", "ETH"],
...     table=vbt.key_dict(BTC="BTC-USD", ETH="ETH-USD"),
...     schema="yf_data"
... ).symbols
['BTC', 'ETH']
```

The same approach can be used to execute custom queries:

```
>>> vbt.DuckDBData.pull(
...     ["BTC-USD", "ETH-USD"],
...     query=vbt.key_dict({
...         "BTC-USD": 'SELECT * FROM yf_data."BTC-USD"',
...         "ETH-USD": 'SELECT * FROM yf_data."ETH-USD"'
...     })
... ).symbols
['BTC-USD', 'ETH-USD']
```

Apart from querying tables, we can also read CSV, Parquet, and JSON files. The reading behavior here is very similar to the writing behavior in **Data.to_duckdb** in that there are three arguments: `read_format`, `read_path`, and `read_options`. The first argument controls the format of the file(s). The second argument controls the path of the file(s). The third argument controls the options for reading the file(s). When `read_format` is not provided, it gets parsed from the extension of the file. It's then used to call the `read_{format}` function from within the SQL query; for example, `read_parquet` for Parquet.

```
>>> vbt.DuckDBData.pull(read_path="data").symbols
['BTC-USD', 'ETH-USD']

>>> vbt.DuckDBData.pull(
...     ["BTC-USD", "ETH-USD"],
...     read_path=vbt.key_dict({
...         "BTC-USD": "data/BTC-USD.parquet",
...         "ETH-USD": "data/ETH-USD.parquet"
...     })
... ).symbols
['BTC-USD', 'ETH-USD']

>>> vbt.DuckDBData.pull(
...     ["BTC-USD", "ETH-USD"],
...     query=vbt.key_dict({
...         "BTC-USD": "SELECT * FROM read_parquet('data/BTC-USD.parquet')",
...         "ETH-USD": "SELECT * FROM read_parquet('data/ETH-USD.parquet')"
...     })
... ).symbols
['BTC-USD', 'ETH-USD']
```

Updating

CSV & HDF

Tabular data such as CSV and HDF data can be read line by line, which makes possible listening for data updates. The classes **CSVData** and **HDFData** can be updated like every preset data class by keeping track of the last row index in **Data.returned_kwargs**. Whenever an update is triggered, this index is being used as the start row from which the dataset should be read. After the update, the end row is being used as the new last row index.

Let's separately download the data for `BTC-USD` and `ETH-USD`, save them to one HDF file, and read the entire file using **HDFData**:

```
>>> yf_data_btc = vbt.YFData.pull(
...     "BTC-USD",
...     start="2020-01-01",
...     end="2020-01-03"
... )
>>> yf_data_eth = vbt.YFData.pull(
...     "ETH-USD",
...     start="2020-01-03",
...     end="2020-01-05"
... )

>>> yf_data_btc.to_hdf("data.h5", key="yf_data_btc")
>>> yf_data_eth.to_hdf("data.h5", key="yf_data_eth")

>>> hdf_data = vbt.HDFData.pull(["BTC-USD", "ETH-USD"], paths="data.h5")
```

Symbol 2/2

```
>>> hdf_data.close
symbol                      BTC-USD     ETH-USD
Date
2020-01-01 00:00:00+00:00  7200.174316         NaN
2020-01-02 00:00:00+00:00  6985.470215         NaN
2020-01-03 00:00:00+00:00          NaN  134.171707
2020-01-04 00:00:00+00:00          NaN  135.069366
```

Let's look at the last row index in each dataset:

```
>>> hdf_data.returned_kwargs
key_dict({'BTC-USD': {'last_row': 1}, 'ETH-USD': {'last_row': 1}})
```

We see that the third row in each dataset is the new start row (1 row holding the header and 1 row holding the data). Let's append new data to the `BTC-USD` dataset and then update our **HDFData** instance:

```
>>> yf_data_btc = yf_data_btc.update(end="2020-01-06")
>>> yf_data_btc.to_hdf("data.h5", key="yf_data_btc")

>>> hdf_data = hdf_data.update()
>>> hdf_data.close
symbol                       BTC-USD     ETH-USD
Date
2020-01-01 00:00:00+00:00  7200.174316         NaN
2020-01-02 00:00:00+00:00  6985.470215         NaN
2020-01-03 00:00:00+00:00  7344.884277  134.171707
2020-01-04 00:00:00+00:00  7410.656738  135.069366
2020-01-05 00:00:00+00:00  7411.317383         NaN
```

The `BTC-USD` dataset has been updated with 3 new data points while the `ETH-USD` dataset hasn't been updated. This is reflected in the last row index:

```
>>> hdf_data.returned_kwargs
key_dict({
    'BTC-USD': {'last_row': 4, 'tz_convert': datetime.timezone.utc},
    'ETH-USD': {'last_row': 1, 'tz_convert': datetime.timezone.utc}
})
```

This workflow can be repeated endlessly.

Feather & Parquet

While Feather and Parquet classes don't have any `start` or `end` arguments to select a date range to pull, we can still load the entire data and append the difference to the currently stored data. This operation isn't too costly because reading such data format is very efficient. But starting from newer versions of Pandas, we can also filter partitions using the `filters` argument (only Parquet format supports this feature):

```
>>> parquet_data = vbt.ParquetData.pull(
...     "BNB-USD",
...     filters=[("group", "<", "2020-01-03")]
... )
>>> parquet_data.close
Date
2020-01-01 00:00:00+00:00    13.689083
2020-01-02 00:00:00+00:00    13.027011
Name: Close, dtype: float64

>>> parquet_data = parquet_data.update(
...     filters=[("group", ">=", "2020-01-03")]
... )
>>> parquet_data.close
Date
2020-01-01 00:00:00+00:00    13.689083
2020-01-02 00:00:00+00:00    13.027011
2020-01-03 00:00:00+00:00    13.660452
2020-01-04 00:00:00+00:00    13.891512
Freq: D, Name: Close, dtype: float64
```

Also, in newer versions of PyArrow, we can use the same argument to select rows:

```
>>> parquet_data = vbt.ParquetData.pull(
...     "BNB-USD",
...     filters=[("Date", "<", vbt.timestamp("2020-01-03", tz="UTC"))]
... )
>>> parquet_data.close
Date
2020-01-01 00:00:00+00:00    13.689083
2020-01-02 00:00:00+00:00    13.027011
Name: Close, dtype: float64

>>> parquet_data = parquet_data.update(
...     filters=[("Date", ">=", vbt.timestamp("2020-01-03", tz="UTC"))]
... )
>>> parquet_data.close
Date
2020-01-01 00:00:00+00:00    13.689083
2020-01-02 00:00:00+00:00    13.027011
2020-01-03 00:00:00+00:00    13.660452
2020-01-04 00:00:00+00:00    13.891512
Freq: D, Name: Close, dtype: float64
```

Important

There's an issue with PyArrow starting from the version 12.0.0 that makes it impossible to filter by timezone-aware timestamps - **https://github.com/apache/arrow/issues/37355**. This issue is said to be resolved by the version 14.0.0.

SQL

The class **SQLData** can be updated in two different ways: using the last row number and using the last index. The first approach works only if we have attached a column with row numbers, the name of this column is known and stored in **Data.returned_kwargs** (which happens automatically), and index-based filtering

(`start` and/or `end`) isn't used. If these conditions are true, this approach will be used by default; it will extract the last row number from the DataFrame and pass it as `start_row`.

```
>>> aapl_data.to_sql("postgresql", attach_row_number=True, if_exists="replace")

>>> sql_data = vbt.SQLData.pull(
...     "AAPL",
...     end_row=5,
...     tz="America/New_York"
... )
>>> sql_data.close
Date
2022-01-03 00:00:00-05:00    180.190964
2022-01-04 00:00:00-05:00    177.904068
2022-01-05 00:00:00-05:00    173.171814
2022-01-06 00:00:00-05:00    170.281021
2022-01-07 00:00:00-05:00    170.449341
Freq: D, Name: Close, dtype: float64

>>> sql_data = sql_data.update(end_row=10)
>>> sql_data.close
Date
2022-01-03 00:00:00-05:00    180.190964
2022-01-04 00:00:00-05:00    177.904068
2022-01-05 00:00:00-05:00    173.171814
2022-01-06 00:00:00-05:00    170.281021
2022-01-07 00:00:00-05:00    170.449341
2022-01-10 00:00:00-05:00    170.469131
2022-01-11 00:00:00-05:00    173.330261
2022-01-12 00:00:00-05:00    173.775726
2022-01-13 00:00:00-05:00    170.469131
2022-01-14 00:00:00-05:00    171.340317
Freq: B, Name: Close, dtype: float64
```

The second approach is enabled if the first approach is disabled and row-based filtering (`start_row` and/or `end_row`) isn't used; it will extract the last index from **Data.last_index** and pass it as `start`, regardless of the index data type.

```
>>> aapl_data.to_sql("postgresql", attach_row_number=False, if_exists="replace")

>>> sql_data = vbt.SQLData.pull(
...     "AAPL",
...     end="2022-01-08",
...     tz="America/New_York"
... )
>>> sql_data.close
Date
2022-01-03 00:00:00-05:00    180.190964
2022-01-04 00:00:00-05:00    177.904068
2022-01-05 00:00:00-05:00    173.171814
2022-01-06 00:00:00-05:00    170.281021
2022-01-07 00:00:00-05:00    170.449341
Freq: D, Name: Close, dtype: float64

>>> sql_data = sql_data.update(end="2022-01-15")
>>> sql_data.close
Date
2022-01-03 00:00:00-05:00    180.190964
2022-01-04 00:00:00-05:00    177.904068
2022-01-05 00:00:00-05:00    173.171814
2022-01-06 00:00:00-05:00    170.281021
2022-01-07 00:00:00-05:00    170.449341
2022-01-10 00:00:00-05:00    170.469131
2022-01-11 00:00:00-05:00    173.330261
2022-01-12 00:00:00-05:00    173.775726
2022-01-13 00:00:00-05:00    170.469131
2022-01-14 00:00:00-05:00    171.340317
Freq: B, Name: Close, dtype: float64
```

If the data was originally pulled using a custom query, both approaches will be disabled and we will have to implement either of the approaches manually.

```
>>> sql_data = vbt.SQLData.pull(
...     "AAPL",
...     query="""
...         SELECT *
...         FROM yf_data."AAPL"
...         WHERE yf_data."AAPL"."Date" >= :start AND yf_data."AAPL"."Date" < :end
...     """,
...     tz="America/New_York",
...     index_col="Date",
...     params={
...         "start": vbt.datetime("2022-01-01", tz="America/New_York"),
...         "end": vbt.datetime("2022-01-08", tz="America/New_York")
...     }
... )
>>> sql_data.close
Date
2022-01-03 00:00:00-05:00    180.190964
2022-01-04 00:00:00-05:00    177.904068
2022-01-05 00:00:00-05:00    173.171814
2022-01-06 00:00:00-05:00    170.281021
2022-01-07 00:00:00-05:00    170.449341
Freq: D, Name: Close, dtype: float64

>>> sql_data = sql_data.update(
```

```
...     params={
...         "start": vbt.datetime("2022-01-08", tz="America/New_York"),
...         "end": vbt.datetime("2022-01-18", tz="America/New_York")
...     }
... )
>>> sql_data.close
Date
2022-01-03 00:00:00-05:00    180.190948
2022-01-04 00:00:00-05:00    177.904068
2022-01-05 00:00:00-05:00    173.171844
2022-01-06 00:00:00-05:00    170.281006
2022-01-07 00:00:00-05:00    170.449326
2022-01-10 00:00:00-05:00    170.469116
2022-01-11 00:00:00-05:00    173.330231
2022-01-12 00:00:00-05:00    173.775742
2022-01-13 00:00:00-05:00    170.469116
2022-01-14 00:00:00-05:00    171.340332
Freq: B, Name: Close, dtype: float64
```

DuckDB

Hint

Previous method (using SQLAlchemy) can be used to update from a DuckDB database as well. For this, you have to install the **duckdb-engine** extension.

To update an existing **DuckDBData** using new data, we can either use the arguments `start` and `end`, or construct a SQL query that returns the desired data range.

```
>>> duckdb_data = vbt.DuckDBData.pull(
...     ["BTC-USD", "ETH-USD"],
...     end="2020-01-03",
...     schema="yf_data"
... )
>>> duckdb_data.close
symbol                       BTC-USD     ETH-USD
Date
2020-01-01 00:00:00+00:00  7200.174316  130.802002
2020-01-02 00:00:00+00:00  6985.470215  127.410179

>>> duckdb_data = duckdb_data.update(end=None)
>>> duckdb_data.close
symbol                       BTC-USD     ETH-USD
Date
2020-01-01 00:00:00+00:00  7200.174316  130.802002
2020-01-02 00:00:00+00:00  6985.470215  127.410179
2020-01-03 00:00:00+00:00  7344.884277  134.171707
2020-01-04 00:00:00+00:00  7410.656738  135.069366
```

Or manually using a custom SQL query:

```
>>> duckdb_data = vbt.DuckDBData.pull(
...     ["BTC-USD", "ETH-USD"],
...     query=vbt.key_dict({
...         "BTC-USD": """
...             SELECT * FROM yf_data."BTC-USD"
...             WHERE "Date" < TIMESTAMP '2020-01-03 00:00:00.000000'
...         """,
...         "ETH-USD": """
...             SELECT * FROM yf_data."ETH-USD"
...             WHERE "Date" < TIMESTAMP '2020-01-03 00:00:00.000000'
...         """
...     })
... )
>>> duckdb_data.close
symbol                       BTC-USD     ETH-USD
Date
2020-01-01 00:00:00+00:00  7200.174316  130.802002
2020-01-02 00:00:00+00:00  6985.470215  127.410179

>>> duckdb_data = duckdb_data.update(
...     query=vbt.key_dict({
...         "BTC-USD": """
...             SELECT * FROM yf_data."BTC-USD"
...             WHERE "Date" >= TIMESTAMP '2020-01-03 00:00:00.000000'
...         """,
...         "ETH-USD": """
...             SELECT * FROM yf_data."ETH-USD"
...             WHERE "Date" >= TIMESTAMP '2020-01-03 00:00:00.000000'
...         """
...     })
... )
>>> duckdb_data.close
symbol                       BTC-USD     ETH-USD
Date
2020-01-01 00:00:00+00:00  7200.174316  130.802002
2020-01-02 00:00:00+00:00  6985.470215  127.410179
2020-01-03 00:00:00+00:00  7344.884277  134.171707
2020-01-04 00:00:00+00:00  7410.656738  135.069366
```

The same but using prepared statements:

```
>>> duckdb_data = vbt.DuckDBData.pull(
...     ["BTC-USD", "ETH-USD"],
...     query=vbt.key_dict({
...         "BTC-USD": """
...             SELECT * FROM yf_data."BTC-USD"
...             WHERE "Date" >= $start AND "Date" < $end
...         """,
...         "ETH-USD": """
...             SELECT * FROM yf_data."ETH-USD"
...             WHERE "Date" >= $start AND "Date" < $end
...         """
...     }),
...     params=dict(
...         start=vbt.datetime("2020-01-01"),
...         end=vbt.datetime("2020-01-03")
...     )
... )
>>> duckdb_data.close
symbol                       BTC-USD     ETH-USD
Date
2020-01-01 00:00:00+00:00  7200.174316  130.802002
2020-01-02 00:00:00+00:00  6985.470215  127.410179

>>> duckdb_data = duckdb_data.update(
...     params=dict(
...         start=vbt.datetime("2020-01-03"),
...         end=vbt.datetime("2020-01-05")
...     )
... )
symbol                       BTC-USD     ETH-USD
Date
2020-01-01 00:00:00+00:00  7200.174316  130.802002
2020-01-02 00:00:00+00:00  6985.470215  127.410179
2020-01-03 00:00:00+00:00  7344.884277  134.171707
2020-01-04 00:00:00+00:00  7410.656738  135.069366
```

Similar to above but now filter based on the (dynamically generated) row number:

```
>>> duckdb_data = vbt.DuckDBData.pull(
...     ["BTC-USD", "ETH-USD"],
...     query=vbt.key_dict({
...         "BTC-USD": """
...             SELECT * EXCLUDE (Row) FROM (
...                 SELECT row_number() OVER () AS "Row", * FROM yf_data."BTC-USD"
...             )
...             WHERE Row >= $start_row AND Row < $end_row
...         """,
...         "ETH-USD": """
...             SELECT * EXCLUDE (Row) FROM (
...                 SELECT row_number() OVER () AS "Row", * FROM yf_data."ETH-USD"
...             )
...             WHERE Row >= $start_row AND Row < $end_row
...         """
...     }),
...     params=dict(start_row=1, end_row=3)
... )
>>> duckdb_data.close
symbol                       BTC-USD     ETH-USD
Date
2020-01-01 00:00:00+00:00  7200.174316  130.802002
2020-01-02 00:00:00+00:00  6985.470215  127.410179

>>> duckdb_data = duckdb_data.update(params=dict(start_row=3, end_row=5))
symbol                       BTC-USD     ETH-USD
Date
2020-01-01 00:00:00+00:00  7200.174316  130.802002
2020-01-02 00:00:00+00:00  6985.470215  127.410179
2020-01-03 00:00:00+00:00  7344.884277  134.171707
2020-01-04 00:00:00+00:00  7410.656738  135.069366
```

Python code

Documentation Data

Remote

Data classes that subclass **RemoteData** specialize in pulling (mostly OHLCV) data from remote data sources. In contrast to the classes for locally stored data, they communicate with remote API endpoints and are subject to authentication, authorization, throttling, and other mechanisms that must be taken into account. Also, the amount of data to be fetched is usually not known in advance, and because most data providers have API rate limits and can return only a limited amount of data for each incoming request, there is often a need to iterate over smaller bunches of data and properly concatenate them. Fortunately, vectorbt implements a number of preset data classes that can do all the jobs above automatically.

Arguments

Most remote data classes have the following arguments in common:

Argument	Description
`client`	Client object required to make a request. Usually, the data class implements an additional class method `resolve_client` to instantiate the client based on the keyword arguments in `client_config` if the client is `None`. If the client has been provided, this step is omitted. You don't need to call the method `resolve_client`, it's done automatically.
`client_config`	Keyword arguments used to instantiate the client.
`start`	Start datetime (including). Will be converted into a `datetime.datetime` using **to_tzaware_datetime** and may be further post-processed to fit the format accepted by the data provider.
`end`	End datetime (excluding). Will be converted into a `datetime.datetime` using **to_tzaware_datetime** and may be further post-processed to fit the format accepted by the data provider.
`tz`	Target timezone of the index (internally it becomes `tz_convert`). Also, if `start` and/or `end` don't have a timezone, uses this timezone to localize them.
`timeframe`	Timeframe supplied as a human-readable string (such as `1 day`) consisting of a multiplier (`1`) and a unit (`day`). Will be parsed into a standardized format using **split_freq_str**.
`limit`	Maximum number of data items to return per request.
`delay`	Delay in milliseconds between requests. Helps to deal with API rate limits.
`retries`	Number of retries in case of connectivity and other request-specific issues. Usually, only applied if the data class is capable of collecting data in bunches.
`show_progress`	Whether to show the progress bar of type **ProgressBar**. Usually, only applied if the data class is capable of collecting data in bunches.
`pbar_kwargs`	Keyword arguments used for setting up the progress bar.
`silence_warnings`	Whether to silence all warnings to avoid being overflooded with messages such as in case of timeouts.
`exchange`	Exchange to fetch from in case the data class supports multiple. If the data class supports multiple exchanges, settings can also be defined per exchange!

To get the list of arguments accepted by the fetcher of a remote data class, we can look into the API reference, use the Python's `help` command, or the vectorbt's own helper function **phelp** on the class method **Data.fetch_symbol**, which creates a query for just one symbol and returns a Series/DataFrame:

```
>>> from vectorbtpro import *

>>> vbt.phelp(vbt.CCXTData.fetch_symbol)
CCXTData.fetch_symbol(
    symbol,
    exchange=None,
    exchange_config=None,
    start=None,
    end=None,
    timeframe=None,
    tz=None,
    find_earliest_date=None,
    limit=None,
    delay=None,
    retries=None,
    fetch_params=None,
    show_progress=None,
    pbar_kwargs=None,
    silence_warnings=None,
    return_fetch_method=False
):
    Override `vectorbtpro.data.base.Data.fetch_symbol` to fetch a symbol from CCXT.

    Args:
        symbol (str): Symbol.

            Symbol can be in the `EXCHANGE:SYMBOL` format, in this case `exchange` argument will be ignored.
        exchange (str or object): Exchange identifier or an exchange object.
```

```
            See `CCXTData.resolve_exchange`.
        exchange_config (dict): Exchange config.

            See `CCXTData.resolve_exchange`.
        start (any): Start datetime.

            See `vectorbtpro.utils.datetime_.to_tzaware_datetime`.
        end (any): End datetime.

            See `vectorbtpro.utils.datetime_.to_tzaware_datetime`.
        timeframe (str): Timeframe.

            Allows human-readable strings such as "15 minutes".
        tz (any): Timezone.

            See `vectorbtpro.utils.datetime_.to_timezone`.
        find_earliest_date (bool): Whether to find the earliest date using `CCXTData.find_earliest_date`.
        limit (int): The maximum number of returned items.
        delay (float): Time to sleep after each request (in milliseconds).

            !!! note
                Use only if `enableRateLimit` is not set.
        retries (int): The number of retries on failure to fetch data.
        fetch_params (dict): Exchange-specific keyword arguments passed to `fetch_ohlcv`.
        show_progress (bool): Whether to show the progress bar.
        pbar_kwargs (dict): Keyword arguments passed to `vectorbtpro.utils.pbar.ProgressBar`.
        silence_warnings (bool): Whether to silence all warnings.
        return_fetch_method (bool): Required by `CCXTData.find_earliest_date`.

    For defaults, see `custom.ccxt` in `vectorbtpro._settings.data`.
    Global settings can be provided per exchange id using the `exchanges` dictionary.
```

As we can see, the class **CCXTData** takes the exchange object, the timeframe, the start date, the end date, and other keyword arguments.

> **Hint**
>
> The class method **Data.pull** usually takes the same arguments as **Data.fetch_symbol**.

Settings

But why are all argument values `None`? Remember that `None` has a special meaning and instructs vectorbt to pull the argument's default value from the **global settings**. Particularly, we should look into the settings defined for CCXT, which are located in the dictionary under `custom.ccxt` in **settings.data**:

```
>>> vbt.pprint(vbt.settings.data["custom"]["ccxt"])
Config(
    exchange='binance',
    exchange_config=dict(
        enableRateLimit=True
    ),
    start=None,
    end=None,
    timeframe='1d',
    tz='UTC',
    find_earliest_date=False,
    limit=1000,
    delay=None,
    retries=3,
    show_progress=True,
    pbar_kwargs=dict(),
    fetch_params=dict(),
    exchanges=dict(),
    silence_warnings=False
)
```

Another way to get the settings is by using the method **Data.get_settings**:

```
>>> vbt.pprint(vbt.CCXTData.get_settings(path_id="custom"))
```

> **Hint**
>
> Data classes register two path ids: `base` and `custom`. The id `base` manipulates the settings for the base class **Data**, while the id `custom` manipulates the settings for any subclass of the class **CustomData**.

Using the default arguments will pull the symbol's entire daily history from Binance.

To set any default, we can change the config directly. Let's change the exchange to BitMEX:

```
>>> vbt.settings.data["custom"]["ccxt"]["exchange"] = "bitmex"
```

Even simpler: similarly to how we used the method **Data.get_settings** to get the settings dictionary, let's use the method **Data.set_settings** to set them:

```
>>> vbt.CCXTData.set_settings(path_id="custom", exchange="bitmex")
>>> vbt.settings.data["custom"]["ccxt"]["exchange"]
'bitmex'
```

> **Note**
>
> Overriding keys in the dictionary returned by **Data.get_settings** will have no effect.

What if we messed up? No need to panic! We can reset the settings at any time:

```
>>> vbt.CCXTData.reset_settings(path_id="custom")
>>> vbt.settings.data["custom"]["ccxt"]["exchange"]
'binance'
```

> **Hint**
>
> This won't reset all settings in vectorbt, only those corresponding to this particular class.

Start and end

Specifying dates and times is usually very easy thanks to the built-in datetime parser **to_tzaware_datetime**, which can parse dates and times from various objects, including human-readable strings, such as `1 day ago`:

```
>>> vbt.local_datetime("1 day ago")
datetime.datetime(2023, 8, 28, 20, 57, 49, 77715, tzinfo=tzlocal())
```

Let's illustrate this by fetching the last 10 minutes of the symbols `BTC/USDT` and `ETH/USDT`:

```
>>> ccxt_data = vbt.CCXTData.pull(
...     ["BTC/USDT", "ETH/USDT"],
...     start="10 minutes ago UTC",
...     end="now UTC",
...     timeframe="1m"
... )
```

> **Note**
>
> Different remote data classes may have different symbol notations, such as `BTC/USDT` in CCXT, `BTC-USD` in Yahoo Finance, `BTCUSDT` in Binance, `X:BTCUSD` in Polygon.io, etc.

Symbol 2/2

```
>>> ccxt_data.close
symbol                     BTC/USDT  ETH/USDT
Open time
2023-08-29 18:50:00+00:00  27990.00   1738.22
2023-08-29 18:51:00+00:00  27973.54   1737.43
2023-08-29 18:52:00+00:00  27981.32   1737.53
2023-08-29 18:53:00+00:00  27964.75   1736.64
2023-08-29 18:54:00+00:00  27972.27   1737.15
2023-08-29 18:55:00+00:00  27963.03   1737.10
2023-08-29 18:56:00+00:00  27962.01   1736.70
2023-08-29 18:57:00+00:00  27970.29   1736.82
2023-08-29 18:58:00+00:00  27986.34   1737.00
2023-08-29 18:59:00+00:00  27987.54   1736.80
```

> **Hint**
>
> Dates and times are resolved in **Data.fetch_symbol**. Whenever fetching high-frequency data, make sure to provide an already resolved start and end time using **to_tzaware_datetime**, otherwise, by the time the first symbol has been fetched, the resolved times for the next symbol may have already been changed.

Timeframe

The timeframe format has been standardized across the entire vectorbt codebase, including the preset data classes. This is done by the function **split_freq_str**, which splits a timeframe string into a multiplier and a unit:

```
>>> vbt.dt.split_freq_str("15 minutes")
(15, 'm')

>>> vbt.dt.split_freq_str("daily")
(1, 'D')

>>> vbt.dt.split_freq_str("1wk")
(1, 'W')

>>> vbt.dt.split_freq_str("annually")
(1, 'Y')
```

After the split, each preset data class transforms the resulting multiplier and the unit into the format acceptable by its API. For example, in the class **PolygonData**, the unit `"m"` is translated into `"minute"`, while in the class **AlpacaData** it's translated into `TimeFrameUnit.Minute`. Note, however, that the units such as `"m"` are for internal purposes only and shouldn't be directly used in Pandas. For example, using `"m"` to construct a date offset (for the use in **pandas.DataFrame.resample**) yields a month end, while using it to construct a timedelta yields a minute:

```
>>> from pandas.tseries.frequencies import to_offset
```

```
>>> to_offset("1m")
<MonthEnd>

>>> pd.Timedelta("1m")
Timedelta('0 days 00:01:00')
```

Here, we should use **to_offset** and **to_timedelta** respectively, which are in-house functions capable of classifying many conventional formats:

```
>>> vbt.offset("1m")
<Minute>

>>> vbt.timedelta("1m")
Timedelta('0 days 00:01:00')
```

Let's pull the 30-minute `BTC/USDT` data of the current day:

```
>>> ccxt_data = vbt.CCXTData.pull(
...     "BTC/USDT",
...     start="today midnight UTC",
...     timeframe="30 minutes"
... )
>>> ccxt_data.get()
                              Open      High       Low     Close       Volume
Open time
2023-08-29 00:00:00+00:00  26120.00  26135.20  26092.91  26092.91    244.82583
2023-08-29 00:30:00+00:00  26092.92  26165.99  26084.07  26140.44    372.98168
2023-08-29 01:00:00+00:00  26140.44  26206.24  26105.78  26127.64    595.85951
...                             ...       ...       ...       ...          ...
2023-08-29 18:00:00+00:00  27892.08  27973.14  27835.84  27949.82   1217.90593
2023-08-29 18:30:00+00:00  27949.83  28020.73  27910.70  27971.79   1422.90243
2023-08-29 19:00:00+00:00  27971.78  27982.65  27900.87  27910.01    220.80118
```

Client

Many APIs require a client to make a request. Data classes based on such APIs usually have a class method with the name `resolve_client` for resolving the client, which is called before pulling each symbol. If the client hasn't been provided by the user (`None`), this method creates one automatically based on the config `client_config`. Such a config can contain various things: from API keys to connection parameters. For example, let's take a look at the default client of **BinanceData**:

```
>>> binance_client = vbt.BinanceData.resolve_client()
>>> binance_client
<binance.client.Client at 0x7f893a193af0>
```

To supply information to this client, we can provide keyword arguments directly:

```
>>> binance_client = vbt.BinanceData.resolve_client(
...     api_key="YOUR_KEY",
...     api_secret="YOUR_SECRET"
... )
>>> binance_client
<binance.client.Client at 0x7f89183512e0>
```

Since the client is getting created automatically, we can pass all the client-related information using the argument `client_config` during fetching:

```
>>> binance_data = vbt.BinanceData.pull(
...     "BTCUSDT",
...     client_config=dict(
...         api_key="YOUR_KEY",
...         api_secret="YOUR_SECRET"
...     )
... )
>>> binance_data.get()
                               Open      High       Low     Close  \
Open time
2017-08-17 00:00:00+00:00   4261.48   4485.39   4200.74   4285.08
2017-08-18 00:00:00+00:00   4285.08   4371.52   3938.77   4108.37
2017-08-19 00:00:00+00:00   4108.37   4184.69   3850.00   4139.98
...                             ...       ...       ...       ...
2023-08-27 00:00:00+00:00  26017.38  26182.23  25966.11  26101.77
2023-08-28 00:00:00+00:00  26101.78  26253.99  25864.50  26120.00
2023-08-29 00:00:00+00:00  26120.00  28142.85  25922.00  27895.97

                                  Volume  Quote volume  Trade count  \
Open time
2017-08-17 00:00:00+00:00     795.150377  3.454770e+06         3427
2017-08-18 00:00:00+00:00    1199.888264  5.086958e+06         5233
2017-08-19 00:00:00+00:00     381.309763  1.549484e+06         2153
...                                  ...           ...          ...
2023-08-27 00:00:00+00:00   12099.642160  3.155484e+08       349090
2023-08-28 00:00:00+00:00   22692.626550  5.910089e+08       523057
2023-08-29 00:00:00+00:00   65076.334610  1.768909e+09       968856

                           Taker base volume  Taker quote volume
Open time
2017-08-17 00:00:00+00:00         616.248541        2.678216e+06
2017-08-18 00:00:00+00:00         972.868710        4.129123e+06
2017-08-19 00:00:00+00:00         274.336042        1.118002e+06
...                                      ...                 ...
2023-08-27 00:00:00+00:00        6170.486640        1.609182e+08
```

```
2023-08-28 00:00:00+00:00       10912.405490        2.842262e+08
2023-08-29 00:00:00+00:00       33459.465920        9.098352e+08

[2204 rows x 9 columns]
```

But if you run **BinanceData.resolve_client**, you'd know that it takes time to instantiate a client, and we don't want to wait that long for every single symbol we're attempting to fetch. Thus, a better decision would be instantiating a client manually only once and then passing it via the argument `client`, which will reuse the client and make fetching noticeably faster:

```
>>> binance_data = vbt.BinanceData.pull(
...     "BTCUSDT",
...     client=binance_client
... )
```

Info

This will also enable re-using the client or the client config during updating since passing any argument to the fetcher will store it inside the dictionary **Data.fetch_kwargs**, which is used by the updater.

Warning

But this also means that sharing the data object with anyone may expose your credentials!

To not compromise the security, the recommended approach is to set any credentials and clients globally, as we discussed previously. This won't store them inside the data instance.

```
>>> vbt.BinanceData.set_settings(
...     path_id="custom",
...     client=binance_client
... )
```

Hint

See the API documentation of the particular data class for examples.

Saving

To save any remote data instance, see **this documentation**. In short: pickling is preferred because it also saves all the arguments that were passed to the fetcher, such as the selected timeframe. Those arguments are important when updating - without them, you'd have to provide them manually every time you attempt to update the data.

```
>>> binance_data = vbt.BinanceData.pull(
...     "BTCUSDT",
...     start="today midnight UTC",
...     timeframe="1 hour"
... )
>>> binance_data.save("binance_data")

>>> binance_data = vbt.BinanceData.load("binance_data")
>>> vbt.pprint(binance_data.fetch_kwargs)
symbol_dict(
    BTCUSDT=dict(
        start='today midnight UTC',
        timeframe='1 hour',
        silence_warnings=False
    )
)
```

As we can see, all the arguments were saved along with the data instance. But in a case where we don't plan on updating the data, we can save the arrays themselves across one or multiple CSV files/HDF keys, one per symbol:

```
>>> binance_data.to_csv()  (1)

>>> csv_data = vbt.CSVData.pull("BTCUSDT.csv")
```

(1) The data will be saved into a CSV file with the name `{symbol}.csv`

But what if we wanted to update the data locally stored in a CSV/HDF file? The fetching-related keyword arguments do not include the timeframe and other parameters anymore, they include only those arguments that are important for the data class holding the data - **CSVData**:

```
>>> vbt.pprint(csv_data.fetch_kwargs)
symbol_dict(
    BTCUSDT=dict(
        path=PosixPath('BTCUSDT.csv')
    )
)
```

If we use the update method on this data instance, it would attempt to update using the local data, not using the remote data. To update from a remote endpoint, we need to switch the data class back to the original class, in this case - **BinanceData**. For this, we can use the method **Data.switch_class**, which can additionally clear all the fetching-related and returned keyword arguments that are related to the CSV file:

```
>>> binance_data = csv_data.switch_class(
```

```
...     new_cls=vbt.BinanceData,
...     clear_fetch_kwargs=True,
...     clear_returned_kwargs=True
... )
>>> type(binance_data)
vectorbtpro.data.custom.binance.BinanceData
```

Finally, use the method **Data.update_fetch_kwargs** to update the fetching-related keyword arguments with the timeframe to avoid repeatedly setting it when updating:

```
>>> binance_data = binance_data.update_fetch_kwargs(timeframe="1 hour")
>>> vbt.pprint(binance_data.fetch_kwargs)
symbol_dict(
    BTCUSDT=dict(
        timeframe='1 hour'
    )
)
```

Is there an easier way? Sure! The class methods **RemoteData.from_csv** and **RemoteData.from_hdf** are accessible from all data classes and perform all the operations above automatically:

```
>>> binance_data = vbt.BinanceData.from_csv(
...     "BTCUSDT.csv",
...     fetch_kwargs=dict(timeframe="1 hour")
... )

>>> type(binance_data)
<class 'vectorbtpro.data.custom.binance.BinanceData'>

>>> vbt.pprint(binance_data.fetch_kwargs)
symbol_dict(
    BTCUSDT=dict(
        timeframe='1 hour'
    )
)
```

Updating

Updating a data instance is generally easy:

```
>>> binance_data = binance_data.update()
```

> **Note**
>
> Updating the current data instance always returns a new data instance.

Under the hood, the updater first overrides the start date with the latest date in the index, and then calls the fetcher. That's why we can specify or override any argument originally used in fetching. Also note that it will only pull new data if the end date is not fixed: if we used the end date `2022-01-01` when fetching, it will be used again when updating, thus make sure to set `end` to `"now"` or `"now UTC"`. Let's first fetch the history for the year 2020, and then append the history for the year 2021:

```
>>> binance_data = vbt.BinanceData.pull(
...     "BTCUSDT",
...     start="2020-01-01",
...     end="2021-01-01"
... )
>>> binance_data = binance_data.update(end="2022-01-01")  (1)
>>> binance_data.wrapper.index
DatetimeIndex(['2020-01-01 00:00:00+00:00', '2020-01-02 00:00:00+00:00',
               '2020-01-03 00:00:00+00:00', '2020-01-04 00:00:00+00:00',
               '2020-01-05 00:00:00+00:00', '2020-01-06 00:00:00+00:00',
               ...
               '2021-12-26 00:00:00+00:00', '2021-12-27 00:00:00+00:00',
               '2021-12-28 00:00:00+00:00', '2021-12-29 00:00:00+00:00',
               '2021-12-30 00:00:00+00:00', '2021-12-31 00:00:00+00:00'],
              dtype='datetime64[ns, UTC]', name='Open time', length=731, freq='D')
```

(1) Without overriding, the argument `end` will default to the value passed to the fetcher - `2021-01-01`

From URL

Even though **CSVData** was designed for the local file system, we can apply a couple of tricks to pull remote data with it as well! Remember how it uses **pandas.read_csv**? This function has an argument `filepath_or_buffer`, which can be a URL. All we have to do is to disable the path matching mechanism by setting `match_paths` to False.

Here's an example of pulling S&P 500 index data:

```
>>> url = "https://datahub.io/core/s-and-p-500/r/data.csv"
>>> csv_data = vbt.CSVData.pull(url, match_paths=False)
>>> csv_data.get()
                           SP500  Dividend  Earnings  Consumer Price Index  \
Date
1871-01-01 00:00:00+00:00   4.44      0.26      0.40                 12.46
```

```
1871-02-01 00:00:00+00:00     4.50      0.26      0.40                12.84
1871-03-01 00:00:00+00:00     4.61      0.26      0.40                13.03
...                            ...       ...       ...                  ...
2018-02-01 00:00:00+00:00  2705.16     49.64       NaN              248.99
2018-03-01 00:00:00+00:00  2702.77     50.00       NaN              249.55
2018-04-01 00:00:00+00:00  2642.19       NaN       NaN              249.84

                           Long Interest Rate  Real Price  Real Dividend  \
Date
1871-01-01 00:00:00+00:00                5.32       89.00           5.21
1871-02-01 00:00:00+00:00                5.32       87.53           5.06
1871-03-01 00:00:00+00:00                5.33       88.36           4.98
...                                       ...         ...            ...
2018-02-01 00:00:00+00:00                2.86     2714.34          49.81
2018-03-01 00:00:00+00:00                2.84     2705.82          50.06
2018-04-01 00:00:00+00:00                2.80     2642.19            NaN

                           Real Earnings   PE10
Date
1871-01-01 00:00:00+00:00           8.02    NaN
1871-02-01 00:00:00+00:00           7.78    NaN
1871-03-01 00:00:00+00:00           7.67    NaN
...                                  ...    ...
2018-02-01 00:00:00+00:00            NaN  32.12
2018-03-01 00:00:00+00:00            NaN  31.99
2018-04-01 00:00:00+00:00            NaN  31.19

[1768 rows x 9 columns]
```

AWS S3

Here's another example for AWS S3:

```
>>> import boto3
>>> s3_client = boto3.client("s3")  (1)

>>> symbols = ["BTCUSDT", "ETHUSDT"]
>>> paths = vbt.symbol_dict({
...     s: s3_client.get_object(
...         Bucket="binance",
...         Key=f"data/{s}.csv")["Body"]  (2)
...     for s in symbols
... })
>>> s3_data = vbt.CSVData.pull(symbols, paths=paths, match_paths=False)  (3)
>>> s3_data.close
symbol                      BTCUSDT  ETHUSDT
Open time
2017-08-17 00:00:00+00:00   4285.08   302.00
2017-08-18 00:00:00+00:00   4108.37   293.96
2017-08-19 00:00:00+00:00   4139.98   290.91
2017-08-20 00:00:00+00:00   4086.29   299.10
2017-08-21 00:00:00+00:00   4016.00   323.29
...                             ...      ...
2022-02-14 00:00:00+00:00  42535.94  2929.75
2022-02-15 00:00:00+00:00  44544.86  3183.52
2022-02-16 00:00:00+00:00  43873.56  3122.30
2022-02-17 00:00:00+00:00  40515.70  2891.87
2022-02-18 00:00:00+00:00  39892.83  2768.74

[1647 rows x 2 columns]
```

(1) See **Python, Boto3, and AWS S3: Demystified**

(2) Adapt the code to your own bucket and keys

(3) Each path in `paths` will become `filepath_or_buffer`

We could have loaded both datasets using **pandas.read_csv** itself, but wrapping them with **CSVData** allows us to take advantage of the vectorbt's powerful **Data** class, for example, to update the remote datasets whenever new data points arrive - a true 💎

Python code

Documentation Data

Synthetic

Synthetic data is data that might have been generated by financial markets but was not. Synthetic price and return data address the financial small data problem and have numerous uses, including testing new investment strategies and feeding data-hungry ML models. They also help us to detect behavior and outlier discrepancies between real and mimicked markets. For example, if our model performs well on a subset of real-world data, we can check it against synthetic data to find out whether we introduced the look-ahead bias or any other Achilles' heel to our model without knowing it.

To assist us in generating synthetic data, vectorbt implements the class **SyntheticData**, which takes the start date, the end date, and the frequency, and builds a datetime-like Pandas Index. It then calls the abstract class method **SyntheticData.generate_key**, which takes the key and the index, generates new data, and returns a Series or a DataFrame ready to be consumed by **Data**. The key here is either a feature or a symbol, depending on the data orientation that the user has chosen. We must override this method and implement our own data generation logic.

> **Note**
>
> If the logic depends on the data orientation (that is, whether features or symbols should be generated), you should be more specific and override **SyntheticData.generate_symbol** and/or **SyntheticData.generate_feature**.

There are two preset classes: **RandomData**, which uses cumulative normally-distributed returns, and **GBMData**, which uses the **Geometric Brownian Motion**. Both generators are very basic but still very interesting to test the models against. One weakness of them is that real asset prices regularly make dramatic moves in response to new information. To account for this, we'll build a data generator based on the **Lévy alpha-stable distribution**!

```
>>> from vectorbtpro import *  (1)
>>> from scipy.stats import levy_stable

>>> def geometric_levy_price(alpha, beta, drift, vol, shape):  (2)
...     _rvs = levy_stable.rvs(alpha, beta,loc=0, scale=1, size=shape)
...     _rvs_sum = np.cumsum(_rvs, axis=0)
...     return np.exp(vol * _rvs_sum + (drift - 0.5 * vol ** 2))

>>> class LevyData(vbt.SyntheticData):  (3)
...
...     _settings_path = dict(custom="data.custom.levy")  (4)
...
...     @classmethod
...     def generate_key(
...         cls,
...         key,
...         index,
...         columns,
...         start_value=None,  (5)
...         alpha=None,
...         beta=None,
...         drift=None,
...         vol=None,
...         seed=None,
...         **kwargs
...     ):
...         start_value = cls.resolve_custom_setting(start_value, "start_value")  (6)
...         alpha = cls.resolve_custom_setting(alpha, "alpha")
...         beta = cls.resolve_custom_setting(beta, "beta")
...         drift = cls.resolve_custom_setting(drift, "drift")
...         vol = cls.resolve_custom_setting(vol, "vol")
...         seed = cls.resolve_custom_setting(seed, "seed")
...         if seed is not None:
...             np.random.seed(seed)
...
...         shape = (len(index), len(columns))
...         out = geometric_levy_price(alpha, beta, drift, vol, shape)
...         out = start_value * out
...         return pd.DataFrame(out, index=index, columns=columns)

>>> LevyData.set_custom_settings(  (7)
...     populate_=True,
...     start_value=100.,
...     alpha=1.68,
...     beta=0.01,
...     drift=0.0,
...     vol=0.01,
...     seed=None
... )
```

(1) Imports `np`, `pd`, `njit`, and `vbt`

(2) Generation function, see **Asset price mimicry**

(3) Subclass **SyntheticData**

(4) Specify the path to global settings for this class and assign it a new identifier "custom". There's one more identifier already registered for us: "base", which points to general settings.

(5) Set most arguments to None to pull their default value from the global settings. You can also hard-code the values here if you've decided to not use global settings.

(6) Access the global settings and if the argument value is None, then replace None with the default value

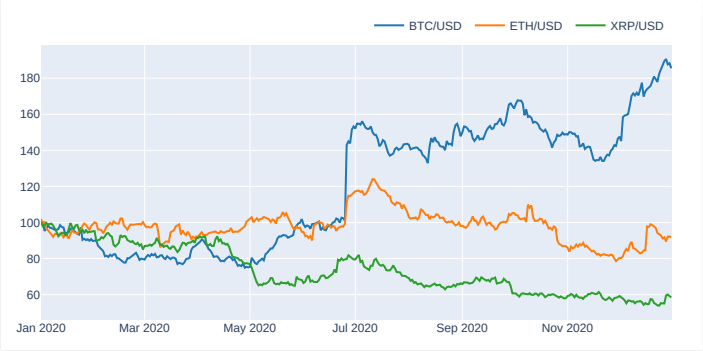
7 Populate the global settings for this class

Let's try it out by generating and plotting the close price of several symbols of data:

```
>>> levy_data = LevyData.pull(
...     "Close",
...     keys_are_features=True,
...     columns=pd.Index(["BTC/USD", "ETH/USD", "XRP/USD"], name="symbol"),
...     start="2020-01-01 UTC",
...     end="2021-01-01 UTC",
...     seed=42)
>>> levy_data.get()
symbol                       BTC/USD     ETH/USD     XRP/USD
2020-01-01 00:00:00+00:00   99.218626  101.893255  100.371131
2020-01-02 00:00:00+00:00  100.062835   99.537102   97.857226
2020-01-03 00:00:00+00:00   95.321467  100.474547   98.246993
2020-01-04 00:00:00+00:00   96.493680   96.455981   99.797874
2020-01-05 00:00:00+00:00   98.489931   95.658733   98.892301
...                               ...         ...         ...
2020-12-27 00:00:00+00:00  189.477849   91.730109   55.055316
2020-12-28 00:00:00+00:00  190.620767   89.452822   59.555616
2020-12-29 00:00:00+00:00  187.641089   92.164802   60.034154
2020-12-30 00:00:00+00:00  188.287168   92.245270   59.188719
2020-12-31 00:00:00+00:00  185.500114   91.701142   58.443060

[366 rows x 3 columns]

>>> levy_data.get().vbt.plot().show()
```



Well done! We've built our own data mimicker that simulates sudden large changes in price.

Python code

Documentation Data

Scheduling

Most data sources aren't sitting idle: they steadily generate new data. To keep up with new information, we can utilize a schedule manager (or even the simplest while-loop) to periodically run jobs related to data capturing and manipulation.

Updating

We can schedule data updates easily using **DataUpdater**, which takes a data instance of type **Data** and a schedule manager of type **ScheduleManager**, and periodically triggers an update that replaces the old data instance with the new one. We can then access the new instance under **DataUpdater.data**. The update happens in the method **DataUpdater.update**, which can be overridden and used to run some custom logic when new data arrives. Since the updater class subclasses **Configured**, it also takes care of replacing its config once `data` changes.

Important

It's one of the few classes in vectorbt that aren't read-only. Do not rely on caching inside it!

Let's use this simple but powerful class to update and plot the last 10 minutes of a Binance ticker, every 10 seconds, for 5 minutes. First, we will pull the latest 10 minutes of data:

```
>>> from vectorbtpro import *

>>> data = vbt.BinanceData.pull(
...     "BTCUSDT",
...     start="10 minutes ago UTC",
...     end="now UTC",
...     timeframe="1m"
... )

>>> data.close
Open time
2022-02-19 20:09:00+00:00    40005.78
2022-02-19 20:10:00+00:00    40001.80
2022-02-19 20:11:00+00:00    40006.45
2022-02-19 20:12:00+00:00    40003.68
2022-02-19 20:13:00+00:00    40022.24
2022-02-19 20:14:00+00:00    40026.73
2022-02-19 20:15:00+00:00    40048.88
2022-02-19 20:16:00+00:00    40044.92
2022-02-19 20:17:00+00:00    40044.03
2022-02-19 20:18:00+00:00    40049.93
Freq: T, Name: Close, dtype: float64
```

Then, we'll subclass **DataUpdater** to accept the figure and update it together with the data. Moreover, to not miss anything visually, after each update, we will append the figure's PNG image to a GIF file:

```
>>> import imageio.v2 as imageio

>>> class OHLCFigUpdater(vbt.DataUpdater):
...     _expected_keys = None
...
...     def __init__(self, data, fig, writer=None, display_last=None,
...                  stop_after=None, **kwargs):
...         vbt.DataUpdater.__init__(  (1)
...             self,
...             data,
...             writer=writer,  (2)
...             display_last=display_last,
...             stop_after=stop_after,
...             **kwargs
...         )
...
...         self._fig = fig
...         self._writer = writer
...         self._display_last = display_last
...         self._stop_after = stop_after
...         self._start_dt = vbt.utc_datetime()  (3)
...
...     @property  (4)
...     def fig(self):
...         return self._fig
...
...     @property
...     def writer(self):
...         return self._writer
...
...     @property
...     def display_last(self):
...         return self._display_last
...
...     @property
...     def stop_after(self):
...         return self._stop_after
```

```
...
...     @property
...     def start_dt(self):
...         return self._start_dt
...
...     def update(self, **kwargs):
...         vbt.DataUpdater.update(self, **kwargs)  # (5)
...
...         df = self.data.get()
...         if self.display_last is not None:
...             df = df[df.index[-1] - self.display_last:]  # (6)
...
...         trace = self.fig.data[0]
...         with self.fig.batch_update():
...             trace.x = df["Close"].index  # (7)
...             trace.open = df["Open"].values
...             trace.high = df["High"].values
...             trace.low = df["Low"].values
...             trace.close = df["Close"].values
...
...         if self.writer is not None:
...             fig_data = imageio.imread(self.fig.to_image(format="png"))
...             self.writer.append_data(fig_data)  # (8)
...
...         if self.stop_after is not None:
...             now_dt = vbt.utc_datetime()
...             if now_dt - self.start_dt >= self.stop_after:
...                 raise vbt.CancelledError  # (9)
```

1. Call the constructor of `DataUpdater`
2. Pass all class-specific keyword arguments to include them in the config
3. Register the start time
4. Properties prevent the user (and the program) from overwriting the object, which is some kind of convention in vectorbt
5. Call `DataUpdater.update`, otherwise, the data won't update!
6. Get the OHLC data within a specific time period (optional)
7. Update the data of the trace (see Candlestick Charts)
8. Append the data of the figure to the GIF file (optional)
9. Stop once the job has run for a specific amount of time by throwing CancelledError (optional)

Don't forget to enable logging (if desirable):

```
>>> import logging

>>> logging.basicConfig(level = logging.INFO)
```

Finally, run the job of `OHLCFigUpdater` every 10 seconds:

```
>>> fig = data.plot(ohlc_type="candlestick", plot_volume=False)
>>> fig  # (1)

>>> with imageio.get_writer("ohlc_fig_updater.gif", duration=250, loop=0) as writer:  # (2)
...     ohlc_fig_updater = OHLCFigUpdater(
...         data=data,
...         fig=fig,
...         writer=writer,
...         display_last=pd.Timedelta(minutes=10),
...         stop_after=pd.Timedelta(minutes=5)
...     )
...     ohlc_fig_updater.update_every(10)  # (3)
INFO:vectorbtpro.utils.schedule_:Starting schedule manager with jobs [Every 10 seconds do update() (last run: [never], next run: 2022-02-19 21:18:38)]
INFO:vectorbtpro.data.updater:New data has 10 rows from 2022-02-19 20:09:00+00:00 to 2022-02-19 20:18:00+00:00
INFO:vectorbtpro.data.updater:New data has 10 rows from 2022-02-19 20:09:00+00:00 to 2022-02-19 20:18:00+00:00
INFO:vectorbtpro.data.updater:New data has 11 rows from 2022-02-19 20:09:00+00:00 to 2022-02-19 20:19:00+00:00
INFO:vectorbtpro.data.updater:New data has 11 rows from 2022-02-19 20:09:00+00:00 to 2022-02-19 20:19:00+00:00
INFO:vectorbtpro.data.updater:New data has 11 rows from 2022-02-19 20:09:00+00:00 to 2022-02-19 20:19:00+00:00
INFO:vectorbtpro.data.updater:New data has 11 rows from 2022-02-19 20:09:00+00:00 to 2022-02-19 20:19:00+00:00
INFO:vectorbtpro.data.updater:New data has 11 rows from 2022-02-19 20:09:00+00:00 to 2022-02-19 20:19:00+00:00
INFO:vectorbtpro.data.updater:New data has 12 rows from 2022-02-19 20:09:00+00:00 to 2022-02-19 20:20:00+00:00
INFO:vectorbtpro.data.updater:New data has 12 rows from 2022-02-19 20:09:00+00:00 to 2022-02-19 20:20:00+00:00
INFO:vectorbtpro.data.updater:New data has 12 rows from 2022-02-19 20:09:00+00:00 to 2022-02-19 20:20:00+00:00
INFO:vectorbtpro.data.updater:New data has 12 rows from 2022-02-19 20:09:00+00:00 to 2022-02-19 20:20:00+00:00
INFO:vectorbtpro.data.updater:New data has 12 rows from 2022-02-19 20:09:00+00:00 to 2022-02-19 20:20:00+00:00
INFO:vectorbtpro.data.updater:New data has 13 rows from 2022-02-19 20:09:00+00:00 to 2022-02-19 20:21:00+00:00
INFO:vectorbtpro.data.updater:New data has 13 rows from 2022-02-19 20:09:00+00:00 to 2022-02-19 20:21:00+00:00
INFO:vectorbtpro.data.updater:New data has 13 rows from 2022-02-19 20:09:00+00:00 to 2022-02-19 20:21:00+00:00
INFO:vectorbtpro.data.updater:New data has 13 rows from 2022-02-19 20:09:00+00:00 to 2022-02-19 20:21:00+00:00
INFO:vectorbtpro.data.updater:New data has 13 rows from 2022-02-19 20:09:00+00:00 to 2022-02-19 20:21:00+00:00
INFO:vectorbtpro.data.updater:New data has 14 rows from 2022-02-19 20:09:00+00:00 to 2022-02-19 20:22:00+00:00
INFO:vectorbtpro.data.updater:New data has 14 rows from 2022-02-19 20:09:00+00:00 to 2022-02-19 20:22:00+00:00
INFO:vectorbtpro.data.updater:New data has 14 rows from 2022-02-19 20:09:00+00:00 to 2022-02-19 20:22:00+00:00
INFO:vectorbtpro.data.updater:New data has 14 rows from 2022-02-19 20:09:00+00:00 to 2022-02-19 20:22:00+00:00
INFO:vectorbtpro.data.updater:New data has 14 rows from 2022-02-19 20:09:00+00:00 to 2022-02-19 20:22:00+00:00
INFO:vectorbtpro.data.updater:New data has 15 rows from 2022-02-19 20:09:00+00:00 to 2022-02-19 20:23:00+00:00
INFO:vectorbtpro.data.updater:New data has 15 rows from 2022-02-19 20:09:00+00:00 to 2022-02-19 20:23:00+00:00
INFO:vectorbtpro.data.updater:New data has 15 rows from 2022-02-19 20:09:00+00:00 to 2022-02-19 20:23:00+00:00
INFO:vectorbtpro.data.updater:New data has 15 rows from 2022-02-19 20:09:00+00:00 to 2022-02-19 20:23:00+00:00
INFO:vectorbtpro.utils.schedule_:Stopping schedule manager
```

1. Run these two lines in a separate cell to see the updates in real time
2. One frame in 250 milliseconds (i.e., 4 frames per second) and loop indefinitely
3. Using DataUpdater.update_every

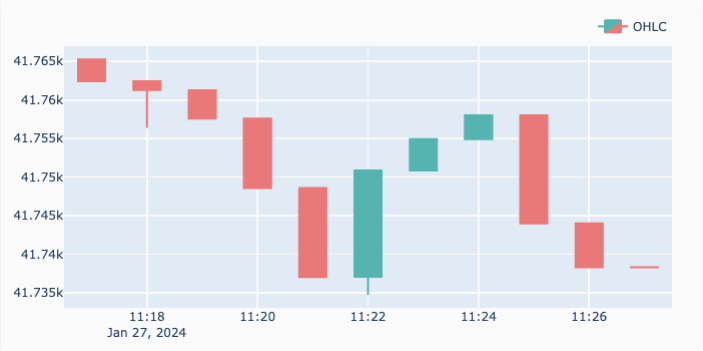
To stop earlier, simply interrupt the execution.

> **Hint**
>
> To run the job in the background, set `in_background` to True. The execution can then be manually stopped by calling `ohlc_fig_updater.schedule_manager.stop()`.

After the data updater has finished running, we can access the entire data:

```
>>> ohlc_fig_updater.data.close
Open time
2022-02-19 20:09:00+00:00    40005.78
2022-02-19 20:10:00+00:00    40001.80
2022-02-19 20:11:00+00:00    40006.45
2022-02-19 20:12:00+00:00    40003.68
2022-02-19 20:13:00+00:00    40022.24
2022-02-19 20:14:00+00:00    40026.73
2022-02-19 20:15:00+00:00    40048.88
2022-02-19 20:16:00+00:00    40044.92
2022-02-19 20:17:00+00:00    40044.03
2022-02-19 20:18:00+00:00    40045.36
2022-02-19 20:19:00+00:00    40047.68
2022-02-19 20:20:00+00:00    40036.74
2022-02-19 20:21:00+00:00    40037.69
2022-02-19 20:22:00+00:00    40039.92
2022-02-19 20:23:00+00:00    40041.62
Freq: T, Name: Close, dtype: float64
```

And here's the produced GIF:



> **Hint**
>
> The smallest time unit of ScheduleManager is a second. For high-precision job scheduling, use a loop with a timer.

Saving

Regular updates with DataUpdater will keep the entire data in memory at any time. But what if we don't need to access the entire data? What if our main objective is to collect as much data as possible from an exchange and write each update directly to disk in a tabular format instead of processing it? This way, we could create one script that writes data updates to a file, and another script that regularly reads that file and performs a job. Moreover, this would make collecting data resilient to errors since we now persist every new bunch of data right away.

If this sounds good, then the DataSaver class is an ideal candidate for such sort of tasks. It subclasses the DataUpdater class and defines additional abstract methods DataSaver.init_save_data and DataSaver.save_data, which should take care of saving the existing data and each new bunch of data respectively into a file.

The workflow of DataSaver is simple. First, it takes a data instance `data` with some initially fetched data. Whenever we call DataSaver.update_every with `init_save=True`, it saves that data into a file using DataSaver.init_save_data. Once the existing data has been persisted to disk, with each new call of DataSaver.update, it pulls the next data update using Data.update with `concat=False` to avoid keeping the existing data in memory. It then calls DataSaver.save_data to **append** the new data to a file. This repeats either until the end of days or until the program gets interrupted by the user or system.

There are two preset subclasses of DataSaver:

1. CSVDataSaver for writing data updates using Data.to_csv
2. HDFDataSaver for writing data updates using Data.to_hdf

Let's pull 1-minute `BTCUSDT` data from Binance and write it to a CSV file, every 10 seconds:

```
>>> data = vbt.BinanceData.pull(
...     "BTCUSDT",
...     start="10 minutes ago UTC",
```

```
...     end="now UTC",
...     timeframe="1m"
... )

>>> csv_saver = vbt.CSVDataSaver(data)
>>> csv_saver.update_every(10, init_save=True)  (1)
INFO:vectorbtpro.data.saver:Saved initial 10 rows from 2022-02-21 23:25:00+00:00 to 2022-02-21 23:34:00+00:00
INFO:vectorbtpro.utils.schedule_:Starting schedule manager with jobs [Every 10 seconds do update() (last run: [never], next run: 2022-02-22
00:34:55)]
INFO:vectorbtpro.data.saver:Saved 1 rows from 2022-02-21 23:34:00+00:00 to 2022-02-21 23:34:00+00:00
INFO:vectorbtpro.data.saver:Saved 2 rows from 2022-02-21 23:34:00+00:00 to 2022-02-21 23:35:00+00:00
INFO:vectorbtpro.data.saver:Saved 1 rows from 2022-02-21 23:35:00+00:00 to 2022-02-21 23:35:00+00:00
INFO:vectorbtpro.data.saver:Saved 1 rows from 2022-02-21 23:35:00+00:00 to 2022-02-21 23:35:00+00:00
INFO:vectorbtpro.data.saver:Saved 1 rows from 2022-02-21 23:35:00+00:00 to 2022-02-21 23:35:00+00:00
```

(1) Use `init_save` to instruct vectorbt to save the initial data before updating

Important

If the initial data isn't on disk yet, pass `init_save=True` to save it first. Otherwise, only updates that follow will be saved!

Note

Don't forget to set up logging as we did in the previous example to see log messages.

Let's interrupt the execution here and throw a look at the data in `csv_saver`:

```
INFO:vectorbtpro.utils.schedule_:Stopping schedule manager

>>> csv_saver.data.close
Open time
2022-02-21 23:35:00+00:00    37185.95
Name: Close, dtype: float64
```

As we can see, in contrast to the data updater we used previously, the data saver keeps only the latest bunch of received data in memory that is necessary for the next update. All the previously fetched data is now stored in a CSV file. Let's take a look:

```
>>> pd.read_csv("BTCUSDT.csv", index_col=0, parse_dates=True)["Close"]  (1)
Open time
2022-02-21 23:25:00+00:00    37247.29
2022-02-21 23:26:00+00:00    37296.44
2022-02-21 23:27:00+00:00    37190.16
2022-02-21 23:28:00+00:00    37135.50
2022-02-21 23:29:00+00:00    37186.11
2022-02-21 23:30:00+00:00    37056.19
2022-02-21 23:31:00+00:00    37079.47
2022-02-21 23:32:00+00:00    37181.49
2022-02-21 23:33:00+00:00    37288.48
2022-02-21 23:34:00+00:00    37209.83
2022-02-21 23:34:00+00:00    37240.21
2022-02-21 23:34:00+00:00    37245.64
2022-02-21 23:35:00+00:00    37248.12
2022-02-21 23:35:00+00:00    37213.55
2022-02-21 23:35:00+00:00    37168.76
2022-02-21 23:35:00+00:00    37185.95
Name: Close, dtype: float64
```

(1) Use Pandas to see the file without post-processing

There are many duplicated index entries, how so? Remember that whenever we request an update, we not only try to fetch new data points but also to update the latest existing ones. When we request 10 updates during a single 1-minute candle, we will get 10 different data points with the same timestamp. Overriding any data point in a CSV file is very inefficient since you need to traverse the entire file just to remove a line. Thus, new data gets appended to a file as it is, and whenever we want to fetch the entire data, **CSVData** will remove any duplicates for us:

```
>>> vbt.CSVData.pull("BTCUSDT.csv").close
Open time
2022-02-21 23:25:00+00:00    37247.29
2022-02-21 23:26:00+00:00    37296.44
2022-02-21 23:27:00+00:00    37190.16
2022-02-21 23:28:00+00:00    37135.50
2022-02-21 23:29:00+00:00    37186.11
2022-02-21 23:30:00+00:00    37056.19
2022-02-21 23:31:00+00:00    37079.47
2022-02-21 23:32:00+00:00    37181.49
2022-02-21 23:33:00+00:00    37288.48
2022-02-21 23:34:00+00:00    37245.64
2022-02-21 23:35:00+00:00    37185.95
Freq: T, Name: Close, dtype: float64
```

To clean the CSV file from duplicates, read the data using **CSVData** and write it back:

```
>>> vbt.CSVData.pull("BTCUSDT.csv").to_csv()

>>> pd.read_csv("BTCUSDT.csv", index_col=0, parse_dates=True)["Close"]
Open time
2022-02-21 23:25:00+00:00    37247.29
2022-02-21 23:26:00+00:00    37296.44
2022-02-21 23:27:00+00:00    37190.16
```

```
2022-02-21 23:28:00+00:00    37135.50
2022-02-21 23:29:00+00:00    37186.11
2022-02-21 23:30:00+00:00    37056.19
2022-02-21 23:31:00+00:00    37079.47
2022-02-21 23:32:00+00:00    37181.49
2022-02-21 23:33:00+00:00    37288.48
2022-02-21 23:34:00+00:00    37245.64
2022-02-21 23:35:00+00:00    37185.95
Name: Close, dtype: float64
```

Info

The step above is optional and brings no big benefit other than disk space savings. You should perform it only occasionally, mainly when the CSV file is to be imported into another program for analysis.

The saving process can be resumed at any time:

```
>>> csv_saver.update_every(10)  (1)
INFO:vectorbtpro.utils.schedule_:Starting schedule manager with jobs [Every 10 seconds do update() (last run: [never], next run: 2022-02-22
00:35:55)]
INFO:vectorbtpro.data.saver:Saved 1 rows from 2022-02-21 23:35:00+00:00 to 2022-02-21 23:35:00+00:00
INFO:vectorbtpro.data.saver:Saved 2 rows from 2022-02-21 23:35:00+00:00 to 2022-02-21 23:36:00+00:00
INFO:vectorbtpro.data.saver:Saved 1 rows from 2022-02-21 23:36:00+00:00 to 2022-02-21 23:36:00+00:00
INFO:vectorbtpro.utils.schedule_:Stopping schedule manager
```

(1) We don't need `init_save` here since this data is already on disk

Note

If the data provider offers only a limited time window of high-granular data, avoid stopping the saving process for a prolonged period of time, otherwise, the data will have blanks.

If we want to resume the saving process even after restarting the runtime, it's advisable to pickle and save the data saver to disk as well:

```
>>> csv_saver.save("csv_saver")  (1)
```

(1) Using **Pickleable.save**

We can then continue in a new runtime:

```
>>> import logging
>>> logging.basicConfig(level = logging.INFO)

>>> from vectorbtpro import *
>>> csv_saver = vbt.CSVDataSaver.load("csv_saver")
>>> csv_saver.update_every(10)
INFO:vectorbtpro.utils.schedule_:Starting schedule manager with jobs [Every 10 seconds do update() (last run: [never], next run: 2022-02-22
00:36:45)]
INFO:vectorbtpro.data.saver:Saved 1 rows from 2022-02-21 23:36:00+00:00 to 2022-02-21 23:36:00+00:00
INFO:vectorbtpro.data.saver:Saved 1 rows from 2022-02-21 23:36:00+00:00 to 2022-02-21 23:36:00+00:00
INFO:vectorbtpro.data.saver:Saved 2 rows from 2022-02-21 23:36:00+00:00 to 2022-02-21 23:37:00+00:00
INFO:vectorbtpro.utils.schedule_:Stopping schedule manager
```

How do we specify where exactly the data should be stored? **DataSaver** takes two arguments: `save_kwargs` and `init_save_kwargs`, which are forwarded to **DataSaver.save_data** and **DataSaver.init_save_data** respectively. For example, in **CSVDataSaver**, those keyword arguments are forwarded down to **Data.to_csv**. Thus, to change the directory path, we can simply do:

```
>>> csv_saver = vbt.CSVDataSaver(
...     save_kwargs=dict(
...         path_or_buf="data",
...         mkdir_kwargs=dict(mkdir=True)
...     )
... )
```

But this is not the only way to provide keyword arguments for saving. If we look closely at the arguments taken by the method **DataUpdater.update_every**, we would again see `save_kwargs` and `init_save_kwargs`, which are forwarded down to their methods. Those arguments have more priority and override the arguments with the same name passed to the constructor. This way, we can change the way the data is saved every time we resume the operation.

The same goes for **HDFDataSaver**.

Python code